

Rapport de Stage

Système de Gestion de l'Internat AIAC

(ResiAIAC)

Amine Sidki

Mohammed Yacine Daher

*Conception et développement d'un système distribué pour la gestion de la résidence
des étudiants à l'AIAC*

Table des matières

1	Étude bibliographique	3
1.1	Introduction	3
1.2	Technologies et Outils Backend	3
1.2.1	Spring Boot	3
1.2.2	PostgreSQL	3
1.2.3	Redis	3
1.2.4	SeaweedFS	4
1.2.5	Keycloak	4
1.2.6	Spring Security & RBAC	4
1.2.7	MapStruct	5
1.2.8	JUnit 5, Mockito & MockMvc	5
1.3	Technologies et Outils Frontend	5
1.3.1	Angular	5
1.3.2	RxJS	5
1.3.3	TailwindCSS	6
1.4	Infrastructure, Déploiement et Concepts	6
1.4.1	Docker & Docker Compose	6
1.4.2	Kubernetes (K8s)	6
1.4.3	GitHub Actions	6
1.4.4	Systèmes Distribués & API REST	6
1.4.5	Gestion d'État (State Management)	6
1.4.6	Mise en Cache (Caching)	6
1.5	Conclusion et Références Bibliographiques	7
2	Étude des opportunités et Analyse de l'existant	8
2.1	Situation Actuelle et Problématique	8
2.2	Synthèse des Problèmes et Opportunités	8
2.3	Analyse Comparative des Outils Existants	8
3	Conception Statistique et Modèle de Données	9
3.1	Diagramme des Classes – Spécifications du Modèle de Données	9
3.2	Modèle Physique de Données (Schéma de Base de Données)	11
4	Architecture Applicative Backend	13
4.1	Organisation en Couches	13
4.2	Séparation DTO / Entité	13
4.3	Gestion des Identifiants Composites	14
4.4	Gestion des Erreurs	14
5	Architecture de Déploiement	14
5.1	Intégration et Déploiement Continu (CI/CD)	15
6	Modélisation des Processus	16
6.1	Dépôt de Documents par l'Étudiant	16
6.2	Traitement et Validation de Dossier par le Responsable	17
6.3	Réservation de Chambre par l'Étudiant	18
6.4	Dépôt d'une Réclamation par l'Étudiant	19
6.5	Suivi de l'Avancement par l'Étudiant	20
6.6	Traitement et Clôture par le Manager de Résidence	21

6.7	Processus Administratif de Libération de Chambre	21
6.8	Gestion Applicative des Profils Utilisateurs	22
6.9	Gestion Fine des Rôles (RBAC)	24
6.10	Consultation des Journaux d'Audit Sécurisés	24
6.11	Dépôt de Documents par l'Étudiant (ouvert)	25
6.12	Traitement et Validation de Dossier par le Responsable (ouvert)	26
6.13	Réservation de Chambre par l'Étudiant (ouvert)	27
6.14	Dépôt de Réclamation par l'Étudiant (ouvert)	28
6.15	Suivi de l'Avancement par l'Étudiant (ouvert)	29
6.16	Traitement et Clôture par le Manager de Résidence (ouvert)	30
7	Scénarios Alternatifs et Cas d'Échec	31
7.1	Échec du Dépôt de Document	31
7.2	Rejet de Document	31
7.3	Chambre Complète (RoomFullException)	32
7.4	Échec de Validation d'Entrée	32
7.5	Accès à la Réclamation d'un Autre Étudiant	33
7.6	Transition d'État	33
7.7	Cas Transversaux : Rejet RBAC	33
8	Perspectives et Périmètre du Projet	35
8.1	Frontend	35
8.2	Contrat d'API et Documentation	35
8.3	Versionnement de Base de Données	35
8.4	Stratégie de Mise en Cache (Redis)	35

1 Étude bibliographique

1.1 Introduction

Cette analyse bibliographique présente un panorama complet des technologies, frameworks, outils et concepts modernes de développement logiciel mis en œuvre dans le cadre du projet **ResiAIAC**. Ce système intègre des composants backend et frontend avancés, des architectures distribuées hautement scalables et des pratiques d'intégration et de déploiement continus (DevOps) de standard industriel.

1.2 Technologies et Outils Backend

1.2.1 Spring Boot

Définition : Spring Boot est un framework Java d'entreprise open-source, conçu sur l'écosystème Spring, qui élimine la complexité de configuration des applications autonomes prêtes pour la production. Il repose sur le paradigme de *"convention plutôt que configuration"* (Convention over Configuration).

Caractéristiques clés :

- **Auto-configuration :** Résolution et instanciation automatiques des beans requis en fonction des dépendances présentes.
- **Serveur Embarqué :** Intégration par défaut de serveurs d'applications (Tomcat, Jetty ou Undertow) éliminant le besoin de serveurs d'applications externes.
- **Gestion des Dépendances :** Utilisation des "starters" Maven/Gradle standardisant les architectures de dépendance.

Intérêt pratique : Permet le développement ultra-rapide de microservices extensibles et d'API REST résilientes et sécurisées.

1.2.2 PostgreSQL

Définition : Système de gestion de base de données relationnelle et objet (SGBDR) open-source de classe entreprise. Il est mondialement reconnu pour son respect rigoureux des standards SQL, sa fiabilité exceptionnelle et sa richesse fonctionnelle.

Caractéristiques clés :

- **Transactions ACID :** Support complet et robuste garantissant l'intégrité absolue des données.
- **Modèles de Données Hybrides :** Gestion native et optimisée du format JSONB permettant d'associer la flexibilité du NoSQL à la rigueur du relationnel.
- **Extensibilité :** Support d'extensions avancées (PostGIS pour les données spatiales, pgvector pour l'IA).

1.2.3 Redis

Définition : Redis (Remote Dictionary Server) est une base de données en mémoire, clé-valeur, ultra-rapide. Elle est principalement exploitée comme cache applicatif, courtier de messages (Message Broker) ou stockage de sessions.

Caractéristiques clés :

- Structures de données hautement optimisées (listes, hashes, sets triés).
- Latence inférieure à la milliseconde grâce au stockage en RAM.
- Persistance configurable (RDB/AOF) pour prévenir la perte accidentelle de données.

1.2.4 SeaweedFS

Définition : Système de stockage distribué d'objets, hautement scalable et efficace pour le stockage de milliards de petits fichiers, inspiré de l'architecture Haystack de Facebook.

Caractéristiques clés :

- Séparation stricte de la gestion des métadonnées (Master Server) et du stockage physique des fichiers (Volume Servers).
- Compatibilité totale avec l'API Amazon S3.
- Réplication native et basculement automatique garantissant une haute disponibilité.

Justification du choix face à MinIO : MinIO a longtemps été la référence de facto du stockage objet auto-hébergé compatible S3. Cependant, à partir de mai 2025, l'éditeur a progressivement retiré les fonctionnalités d'administration de son édition communautaire (console de gestion, gestion des politiques et de la réplication) pour les réserver à son offre commerciale, avant de placer le dépôt open-source en mode maintenance en décembre 2025. Ce repositionnement délibéré vers l'offre entreprise « AIStor » a fragilisé la viabilité de MinIO comme solution open-source pérenne. SeaweedFS a donc été retenu comme l'alternative open-source la plus viable et activement maintenue, conservant une empreinte légère et une compatibilité S3 complète sans dépendre d'un éditeur en phase de repli commercial.

1.2.5 Keycloak

Définition : Solution open-source de gestion des identités et des accès (IAM), assurant l'authentification et l'autorisation via les protocoles standards OAuth2 et OpenID Connect (OIDC).

Caractéristiques clés :

- **Émission de jetons JWT :** Génère des jetons signés contenant les rôles et informations de l'utilisateur authentifié, vérifiés côté backend sans appel systématique au serveur d'identité.
- **Gestion des *clients* et des rôles :** Permet de définir des applications clientes distinctes (backend, outils de test) et une hiérarchie de rôles applicatifs.
- **Déploiement autonome :** Hébergé au sein du cluster Kubernetes aux côtés des services applicatifs, avec sa propre base de données dédiée.

Intérêt pratique : Centralise l'authentification pour un déploiement mono-tenant (une seule école), sans besoin de fédération SSO entre plusieurs fournisseurs d'identité.

1.2.6 Spring Security & RBAC

Définition : Module de l'écosystème Spring assurant l'authentification et le contrôle d'accès aux ressources d'une application. Couplé à Keycloak, il agit en *Resource Server* validant les jetons JWT entrants.

Caractéristiques clés :

- **Contrôle d'accès basé sur les rôles (RBAC) :** Restriction fine des opérations selon le rôle de l'utilisateur (Étudiant, Responsable, Manager, Administrateur).
- **Journalisation d'audit :** Traçabilité des actions sensibles effectuées sur le système, consultable par les administrateurs.
- **Approvisionnement des comptes (Provisioning) :** Création conjointe du compte Keycloak et de l'entité *Utilisateur* au moment de l'admission, afin d'éviter la circulation de données personnelles non chiffrées dans le jeton JWT.

1.2.7 MapStruct

Définition : Générateur de code Java, exécuté à la compilation, produisant des *mappers* pour convertir automatiquement les entités JPA vers leurs Objets de Transfert de Données (DTO) et inversement.

Caractéristiques clés :

- Génération de code à la compilation (pas de réflexion à l'exécution), garantissant des performances proches d'un mapping manuel.
- Réduction du code répétitif (*boilerplate*) associé à la conversion entité/DTO sur chaque endpoint REST.

1.2.8 JUnit 5, Mockito & MockMvc

Définition : Ensemble de bibliothèques de test formant la pile de vérification automatisée du backend : JUnit 5 comme moteur d'exécution, Mockito pour l'isolation des dépendances, AssertJ pour des assertions lisibles, et MockMvc pour tester la couche web sans démarrer de serveur réel.

Caractéristiques clés :

- **Tests de service (unitaires) :** Chaque `ServiceImpl` est testé isolément, dépendances (*repository*, *mapper*) simulées via `@Mock`; les appels statiques de `ResourceFetcher` sont simulés avec `mockStatic` (Mockito 5 / `mockito-inline`).
- **Tests de Controller (tranche web) :** Chaque Controller est testé via `@WebMvcTest` et `MockMvc`, le service associé étant remplacé par `@MockitoBean`; seuls le routage, la (dé)sérialisation JSON et les codes de statut sont vérifiés ici, la logique métier étant déjà couverte par les tests de service.
- **Filtres de sécurité désactivés à ce niveau :** Les tests de Controller démarrent avec `addFilters = false`, l'authentification Keycloak/JWT étant hors périmètre de cette tranche de tests.
- **Couverture des champs énumérés :** Vérification explicite du comportement des champs de type *enum* (ex. `EtatChambre`, `EtatDocument`), y compris leur acceptation actuelle de la valeur `NULL` — point identifié comme perfectible.

Intérêt pratique : Exécution automatisée à chaque contribution via GitHub Actions, avant intégration au code principal.

1.3 Technologies et Outils Frontend

1.3.1 Angular

Définition : Framework applicatif structuré et typé, développé par Google, utilisant TypeScript pour concevoir des applications web monopages (SPA) robustes, performantes et scalables au niveau entreprise.

Caractéristiques clés :

- Architecture modulaire stricte basée sur les composants réutilisables.
- Injection de dépendances native pour une modularité accrue et une testabilité maximale.
- *Two-Way Data Binding* unifiant de manière transparente le modèle et la vue.

1.3.2 RxJS

Définition : Bibliothèque réactive mettant en œuvre le pattern Observer et les flux d'observables pour orchestrer de manière asynchrone les événements et les requêtes de données.

Caractéristiques clés :

- Gestion fluide des requêtes HTTP concurrentes et du temps réel (WebSockets).
- Large palette d'opérateurs de transformation et de filtrage (`switchMap`, `debounceTime`, `combineLatest`).

1.3.3 TailwindCSS

Définition : Framework CSS orienté "utilitaires d'abord" (Utility-First), permettant de concevoir des interfaces graphiques modernes et réactives directement dans le balisage HTML.

Caractéristiques clés :

- Évite la prolifération de feuilles de style complexes et redondantes.
- Optimisation draconienne de la taille des bundles via l'élimination automatique des classes inutilisées (PurgeCSS).

1.4 Infrastructure, Déploiement et Concepts

1.4.1 Docker & Docker Compose

Définition : Docker standardise l'empaquetage des applications et de leur environnement d'exécution complet dans des conteneurs isolés, garantissant le principe *"build once, run anywhere"*. Docker Compose permet la définition multi-conteneur via un unique fichier déclaratif (`compose.yml`).

1.4.2 Kubernetes (K8s)

Définition : Orchestrateur de conteneurs open-source standard de l'industrie, conçu pour automatiser le déploiement, la mise à l'échelle automatique (Auto-scaling) et la gestion des applications conteneurisées.

1.4.3 GitHub Actions

Définition : Plateforme d'automatisation CI/CD intégrée à GitHub pour exécuter des pipelines de tests automatisés, de validation de code (Linter) et de déploiement automatique sur chaque événement Git.

1.4.4 Systèmes Distribués & API REST

Définition : Architecture distribuée structurant les services indépendamment, communiquant via le protocole HTTP sans état (Stateless) en s'appuyant sur les verbes HTTP standards.

1.4.5 Gestion d'État (State Management)

Définition : Centralisation et prévisibilité des données applicatives (par exemple via NgRx) pour assurer la cohérence des vues sur les interfaces utilisateur complexes.

1.4.6 Mise en Cache (Caching)

Définition : Stratégie d'optimisation visant à éliminer les goulots d'étranglement de la base de données en conservant temporairement en RAM les données hautement sollicitées.

1.5 Conclusion et Références Bibliographiques

Le choix de cette pile technologique moderne garantit la création d'une application performante, modulaire et hautement disponible.

- **Spring Boot** : <https://spring.io/projects/spring-boot>
- **PostgreSQL** : <https://www.postgresql.org/docs/>
- **Redis** : <https://redis.io/docs/>
- **SeaweedFS** : <https://github.com/seaweedfs/seaweedfs>
- **Keycloak** : <https://www.keycloak.org/documentation>
- **Spring Security** : <https://docs.spring.io/spring-security/reference/>
- **MapStruct** : <https://mapstruct.org/documentation/>
- **Docker** : <https://docs.docker.com/>
- **Kubernetes** : <https://kubernetes.io/docs/>
- **GitHub Actions** : <https://docs.github.com/en/actions>
- **Angular** : <https://angular.dev/>

2 Étude des opportunités et Analyse de l'existant

2.1 Situation Actuelle et Problématique

La gestion de la résidence universitaire de l'AIAC repose sur un écosystème hétérogène d'outils cloisonnés. L'application de gestion administrative actuelle est fermée aux étudiants, forçant ces derniers à passer par des processus manuels (cahier de réclamations physique) ou des systèmes déconnectés (formulaires BDE pour la réservation des chambres).

2.2 Synthèse des Problèmes et Opportunités

La fragmentation actuelle engendre plusieurs dysfonctionnements majeurs :

1. **Perte de temps et d'efficacité** : La double saisie des données papier vers Excel augmente la charge administrative.
2. **Erreurs humaines** : La transcription manuelle induit des incohérences de données.
3. **Absence de traçabilité** : Complexité de suivi de l'historique et de l'état des réclamations.
4. **Isolement informationnel** : Manque total de visibilité unifiée sur l'occupation de l'internat.

2.3 Analyse Comparative des Outils Existants

Le tableau ci-dessous dresse l'état des lieux des solutions de contournement actuellement utilisées par les acteurs :

Outil Actuel	Acteurs	Objectif du Processus	Limites Identifiées
Papier / Cahier	Étudiant, Responsable	Saisie des dossiers, réclamations	Perte physique, dégradation
Excel	Manager, Responsable	Suivi des réclamations	Erreurs de formules, pas de partage
Téléphone	Logisticiens, Équipes	Prise de décision urgente	Absence d'historique écrit
Email	Collaborateurs	Envoi de pièces justificatives	Risque de non-réponse, classement
WhatsApp	Manager, Personnel	Échanges informels de terrain	Non professionnel, non structuré

TABLE 1 – Cartographie des outils actuels et leurs limites

3 Conception Statistique et Modèle de Données

Cette section pose les fondations de persistance de **ResiAIAC** : le modèle entités-relations qui sous-tend l'ensemble des entités métier décrites plus loin dans ce document. Chaque choix de modélisation (clés composites, cardinalités, contraintes d'unicité) découle directement des règles de gestion de l'internat – occupation des chambres, suivi des promotions, traçabilité des réclamations – plutôt que d'un schéma générique plaqué a posteriori.

3.1 Diagramme des Classes – Spécifications du Modèle de Données

Pour remplacer l'application existante et centraliser l'information, le modèle de données de **ResiAIAC** est structuré autour des entités et relations clés suivantes :

Utilisateur : Représente toute personne accédant au système (Étudiant, Manager, Responsable, Administrateur). Contient l'historique complet de ses réservations, de ses documents administratifs et de ses réclamations.

Document : Pièce justificative déposée par l'étudiant pour son dossier d'admission (carte d'identité, attestation, etc.). Il possède un état de validation (*Aucun*, *En attente*, *Validé*, *Invalide*) et une note sur sa validité.

Chambre : Entité physique caractérisée par sa capacité, son état d'occupation et sa localisation (Étage et Bâtiment). Elle est rattachée aux réservations et réclamations.

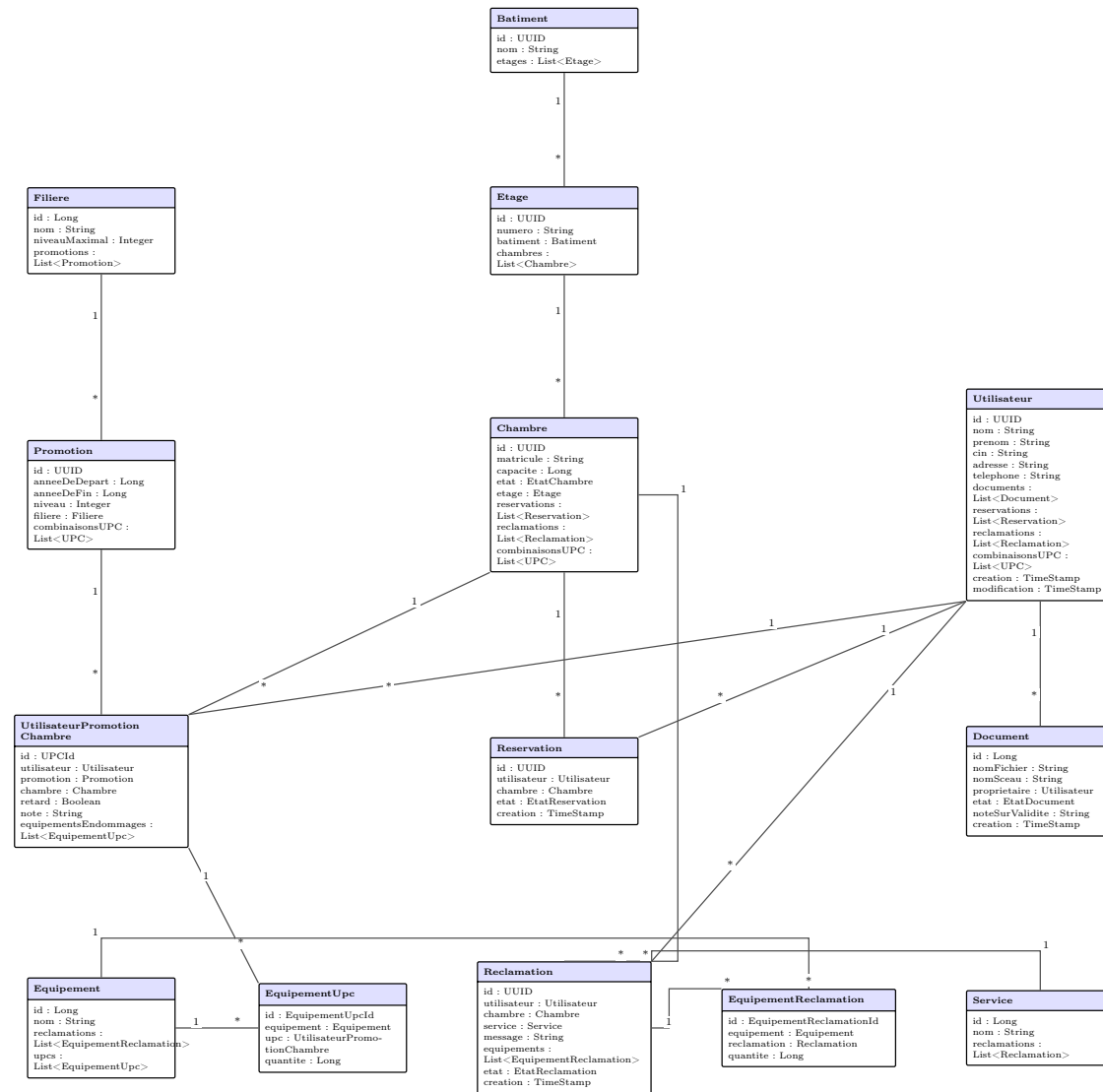
Reservation : Trace historique de l'affectation d'une chambre à un étudiant pour une période donnée.

Reclamation : Ticket ouvert par un étudiant concernant un problème technique dans sa chambre (lié à une entité **Service** et à des équipements défectueux).

UtilisateurPromotionChambre (UPC) : Table d'association majeure modélisant la répartition des étudiants par promotion et par chambre pour assurer un suivi historique rigoureux.

Le diagramme de classes UML ci-après (Figure 3.1) formalise l'ensemble de ces entités, leurs attributs ainsi que les multiplicités des relations qui les lient.

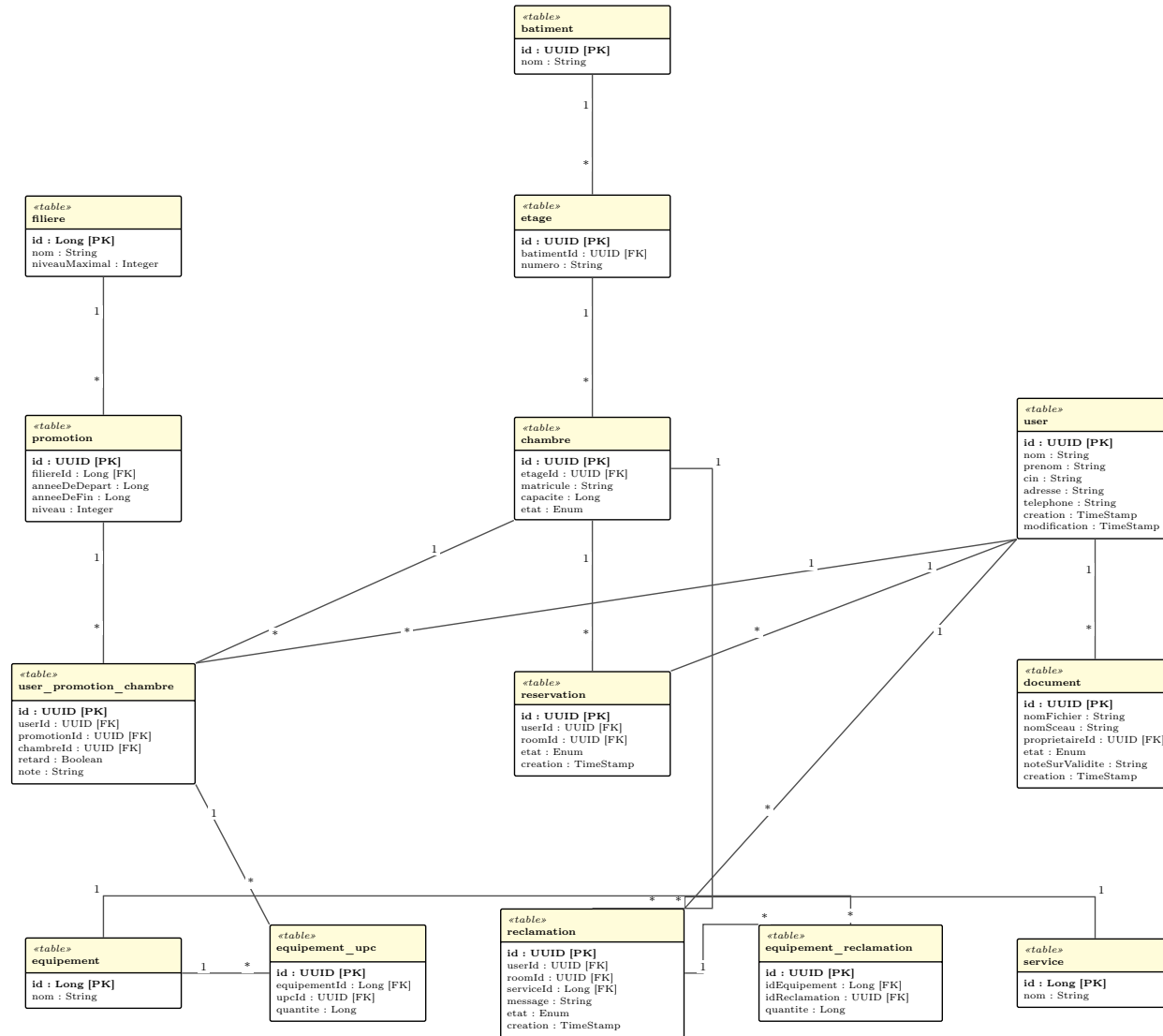
Figure 1 – Diagramme de Classes UML (Modèle Logique)



3.2 Modèle Physique de Données (Schéma de Base de Données)

Le diagramme suivant (Figure 3.2) présente la transposition du modèle logique en un schéma physique de base de données, sous forme de diagramme de classes UML où chaque classe porte le stéréotype « table ». Les attributs identifiants sont marqués **[PK]** (clé primaire) et les références vers d'autres tables **[FK]** (clé étrangère), conformément aux conventions de modélisation physique en UML.

Figure 2 – Diagramme de Classes UML (Modèle Physique / Schéma de Base de Données)



Conclusion. Le modèle de données présenté ci-dessus est directement mappé sur les entités JPA du backend : chaque relation, cardinalité et clé composite décrite plus haut se retrouve dans les migrations générées par Hibernate (§9.3) et dans les DTO manipulés par la couche service, détaillée dans la section suivante.

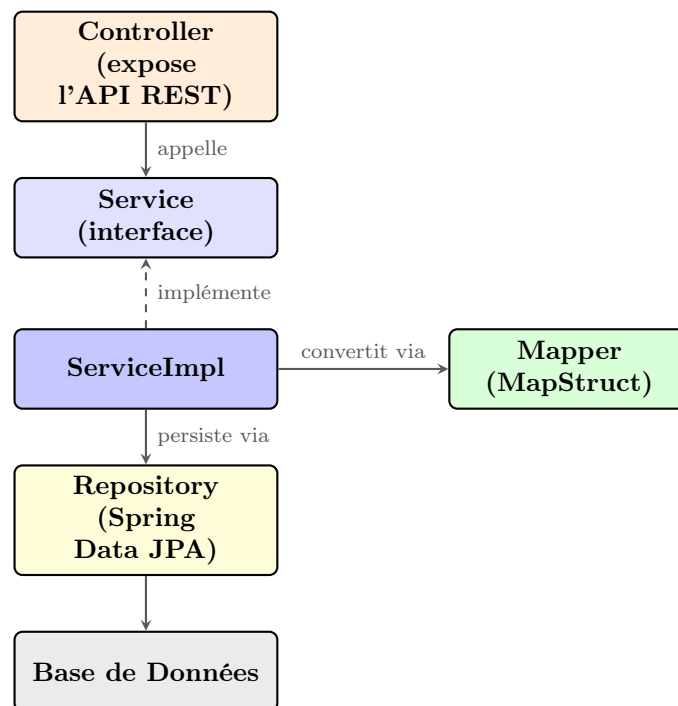
4 Architecture Applicative Backend

Cette section décrit comment le modèle de données ci-dessus est exposé et manipulé côté backend : l'organisation en couches commune aux 14 entités métier, la centralisation de la gestion d'erreurs, et les choix d'infrastructure (stockage objet, IAM) qui traversent l'ensemble de l'application.

4.1 Organisation en Couches

Chaque entité métier du backend (et ses opérations associées) suit rigoureusement la même organisation en couches, illustrée par la Figure 4.1. Le Controller REST ne dépend que de l'**interface** du service, jamais de son implémentation concrète : ce découplage permet de faire évoluer ou remplacer la logique métier d'un service sans modifier son point d'entrée, et facilite l'écriture de tests isolés à chaque couche (voir §1.2.8).

Figure 4 – Organisation en Couches (Backend)



4.2 Séparation DTO / Entité

Les entités JPA ne transitent jamais directement par l'API REST. Chaque entité expose deux objets de transfert distincts, convertis par un *mapper* MapStruct dédié (voir §1.2.7) :

- **XxxDto** : représentation de sortie, retournée par les endpoints de lecture.
- **XxxUpdateRequest** : représentation d'entrée, utilisée pour la création et la mise à jour, validée avant conversion vers l'entité.

Cette séparation évite d'exposer la structure interne des entités (relations JPA, champs techniques) et permet de faire évoluer le modèle de données sans casser le contrat d'API.

4.3 Gestion des Identifiants Composites

Trois entités du modèle de données reposent sur une clé primaire composite, portée par une classe d'identifiant dédiée : `UtilisateurPromotionChambreId`, `EquipementUpcId` et `EquipementReclamationId`. Par convention, les méthodes `getById` et `delete` des services concernés acceptent les composants bruts de la clé (identifiants simples) et reconstituent la clé composite **au sein de la couche service**, plutôt que de laisser le Controller assembler cet objet technique. La méthode `update`, à l'inverse, accepte directement l'identifiant composite déjà construit ; cette asymétrie, actuellement assumée dans l'implémentation, est identifiée comme un point à harmoniser.

4.4 Gestion des Erreurs

La récupération d'une ressource par son identifiant passe systématiquement par l'utilitaire générique `ResourceFetcher.fetchResource(id, repository, resourceType)`, qui centralise la logique *"récupérer ou lever une exception"* et évite sa duplication dans chaque service. Lorsque la ressource est introuvable, une `ResourceNotFoundException` (exception applicative dédiée) est levée.

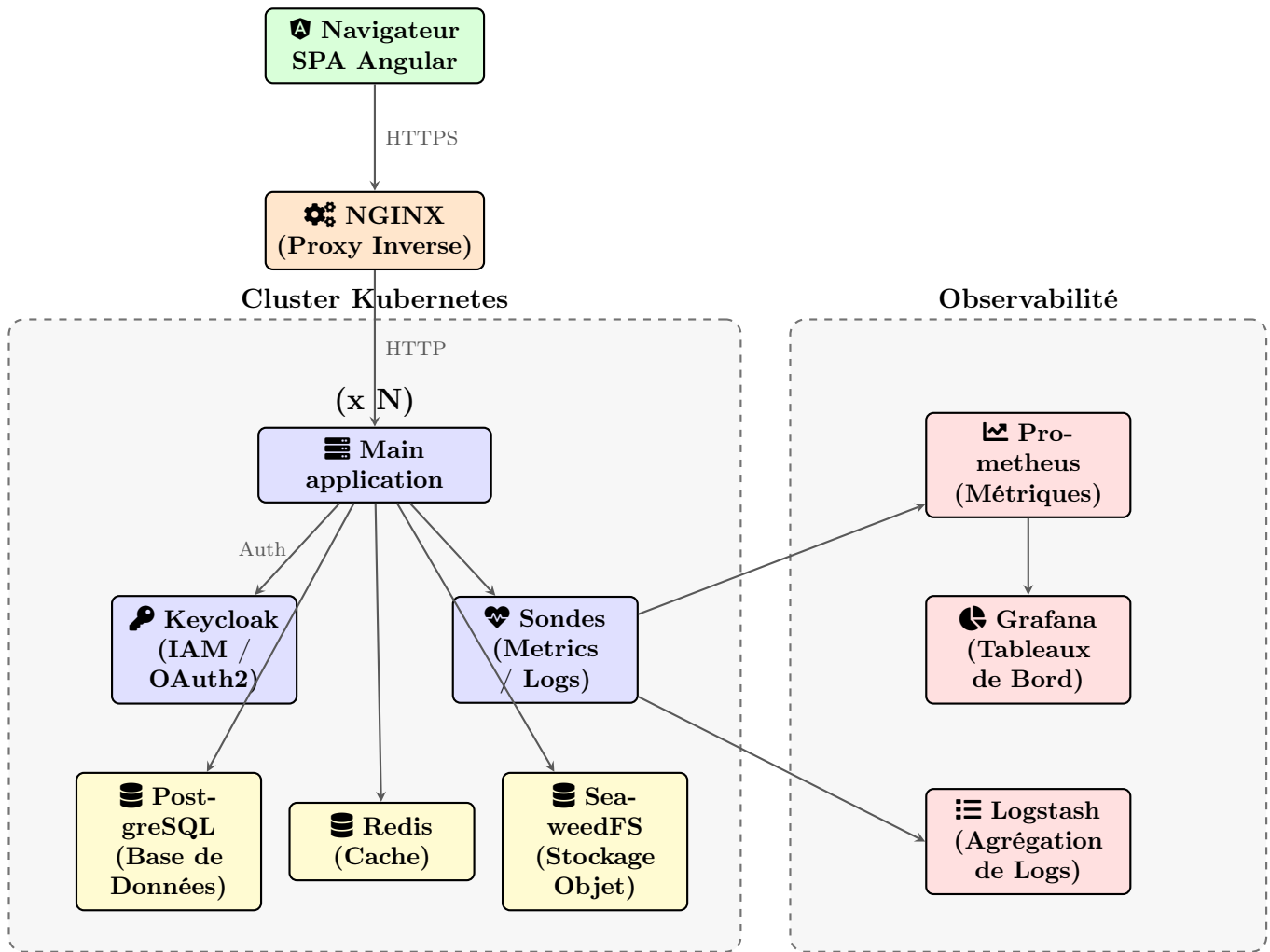
Les erreurs de l'API sont uniformisées à l'échelle de l'application par un `GlobalExceptionHandler` annoté `@RestControllerAdvice`, qui centralise la traduction des exceptions applicatives en réponses HTTP structurées : `ResourceNotFoundException` et `ResourceOwnershipMismatchException` (404), `RoomFullException` et `MethodArgumentNotValidException` (400), avec un gestionnaire générique de repli pour toute exception non anticipée (500). Cette centralisation évite la duplication de la logique de traduction d'erreur dans chaque contrôleur et garantit un format de réponse cohérent (`ErrorResponse`) à travers l'ensemble de l'API.

Conclusion. Cette organisation en couches et cette gestion d'erreurs centralisée constituent le socle sur lequel repose chacune des entités métier détaillées dans les diagrammes de séquence (§7) : tout appel entrant, quel que soit le scénario, traverse le même `Controller` → `Service` → `Repository` et retombe sur le même mécanisme d'erreur en cas d'échec.

5 Architecture de Déploiement

Cette section situe le backend applicatif décrit ci-dessus dans son environnement d'exécution cible : comment les instances Spring Boot, Keycloak et les couches de persistance sont orchestrées, exposées et supervisées en production. Le diagramme de déploiement suivant (Figure 5) illustre l'architecture technique cible de **ResiAIAC**, organisée en couches : couche client (SPA Angular), couche de routage/entrée (NGINX), couche applicative conteneurisée et orchestrée par Kubernetes (services Spring Boot et Keycloak pour l'IAM), couche de persistance et de stockage (PostgreSQL, Redis, SeaweedFS), couche d'observabilité (Prometheus, Grafana, la pile Logstash) et enfin la chaîne d'intégration et de déploiement continu (CI/CD).

Figure 5 – Diagramme de Déploiement de l'Infrastructure ResiAIAC



5.1 Intégration et Déploiement Continu (CI/CD)

La chaîne CI/CD, implémentée via GitHub Actions, applique des portes de qualité strictes à chaque pull request avant intégration :

- **Formatage du code** : vérifié automatiquement par `Spotless` (`mvn spotless:check`) – toute pull request dont le code ne respecte pas le formatage standardisé échoue la vérification.
- **Couverture de tests** : un seuil minimal de 60% de couverture (ratio d'instructions couvertes, mesuré par `JaCoCo`) est appliqué via `mvn verify`. Le périmètre mesuré couvre les contrôleurs et validateurs ; les entités, DTO, mappers, repositories, exceptions, configuration et **la couche d'implémentation des services (service/impl)** en sont exclus par choix – les interfaces de service qu'elles réalisent sont déjà comptabilisées dans le ratio mesuré, ce qui évite un double comptage de la même logique métier.

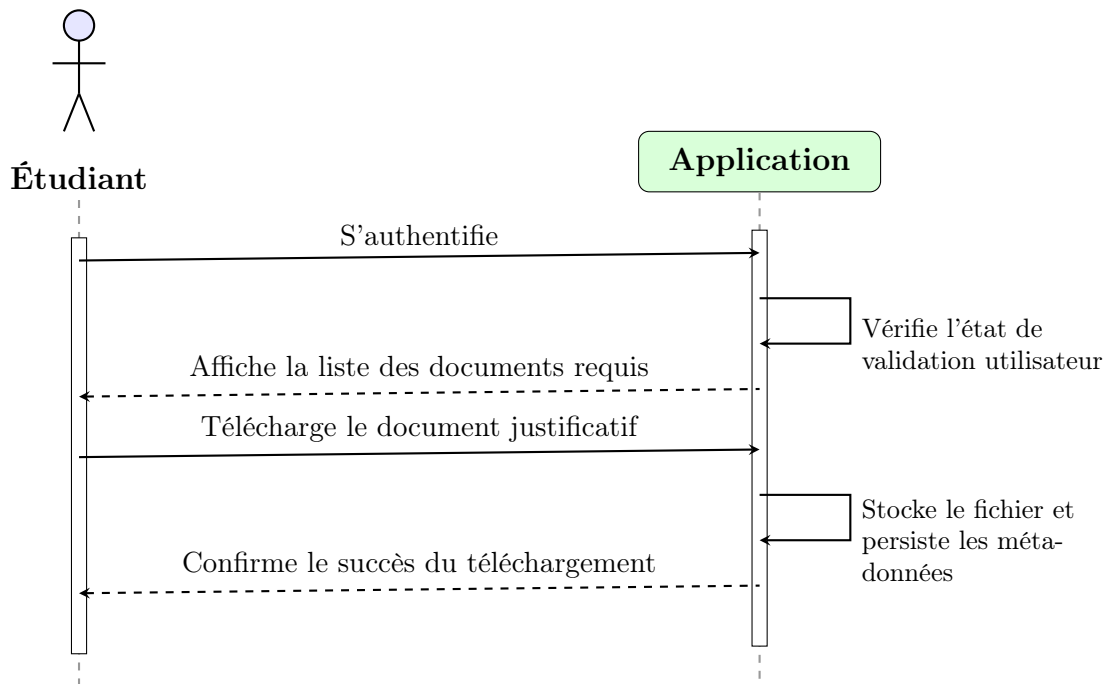
Conclusion. L'architecture de déploiement et la chaîne CI/CD décrites ci-dessus fournissent le cadre d'exécution et de qualité dans lequel s'inscrivent les scénarios métier modélisés dans les deux sections suivantes : chaque appel API illustré par un diagramme de séquence transite par cette infrastructure et respecte les mêmes portes de qualité avant d'être fusionné.

6 Modélisation des Processus

Cette section modélise les principaux scénarios fonctionnels de **ResiAIAC**, d'abord du point de vue de l'utilisateur sous forme de diagrammes de séquence « boîte noire » (seule l'interaction observable entre l'acteur et l'application est représentée), puis, pour les scénarios effectivement implémentés, sous une vue « boîte ouverte » détaillant le cheminement interne réel jusqu'au niveau de l'appel de méthode. Afin de garantir une cohérence graphique et conceptuelle parfaite, l'ensemble des scénarios a été homogénéisé selon le modèle standardisé et direct : **Acteur** → **Application**.

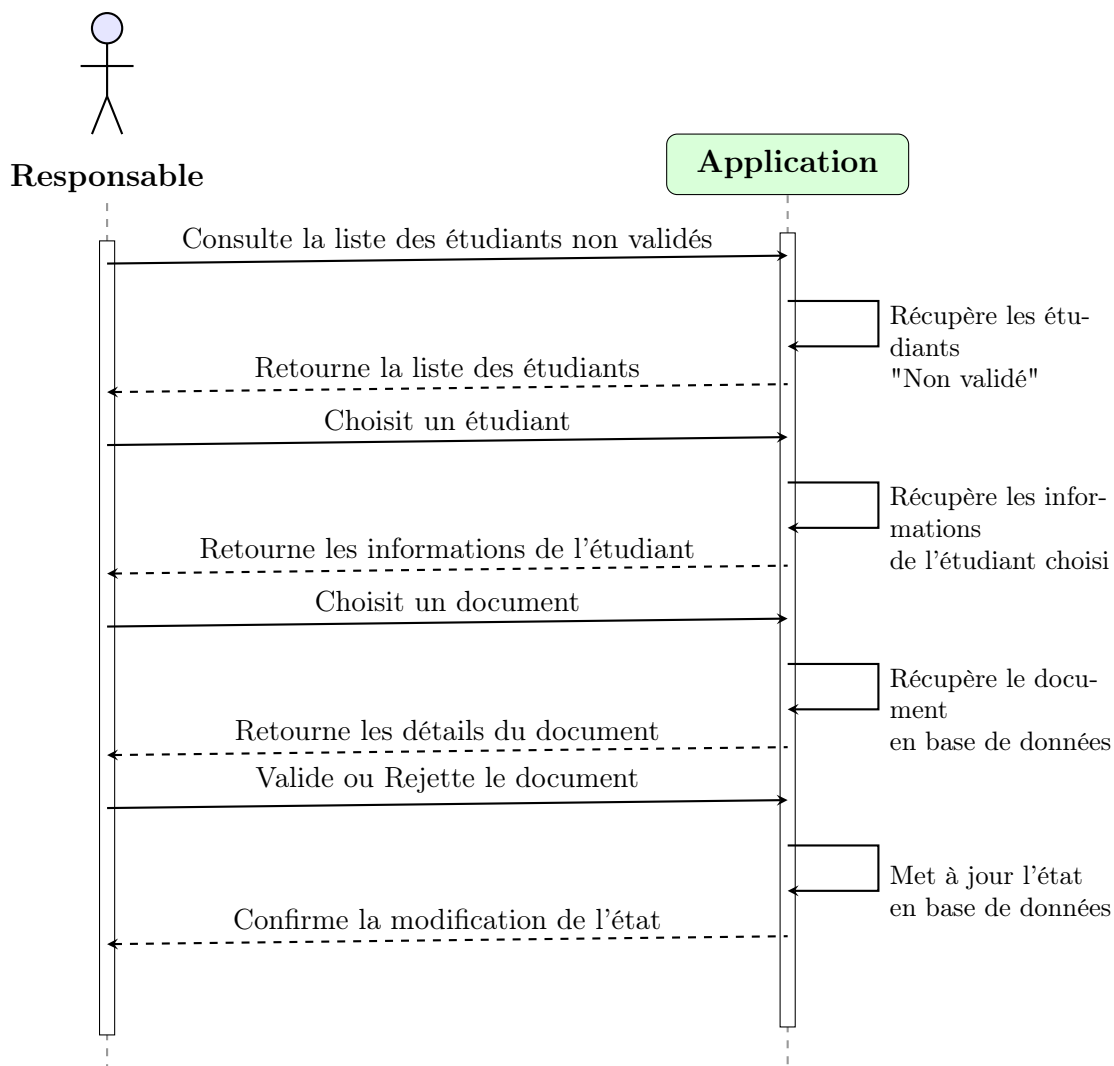
6.1 Dépôt de Documents par l'Étudiant

Ce scénario décrit le flux d'authentification et de téléchargement sécurisé des pièces justificatives de l'étudiant.



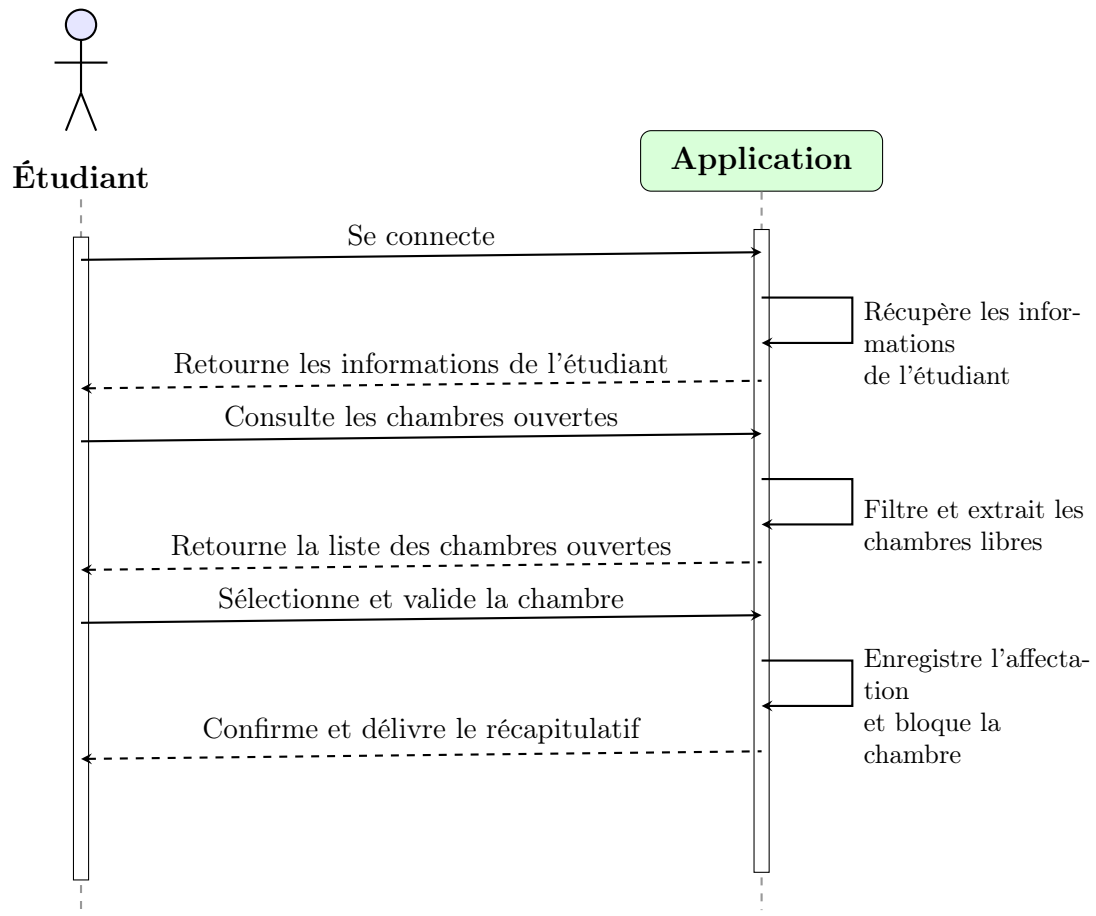
6.2 Traitement et Validation de Dossier par le Responsable

Ce diagramme détaille le flux d'analyse et de validation des pièces justificatives par le responsable d'internat.



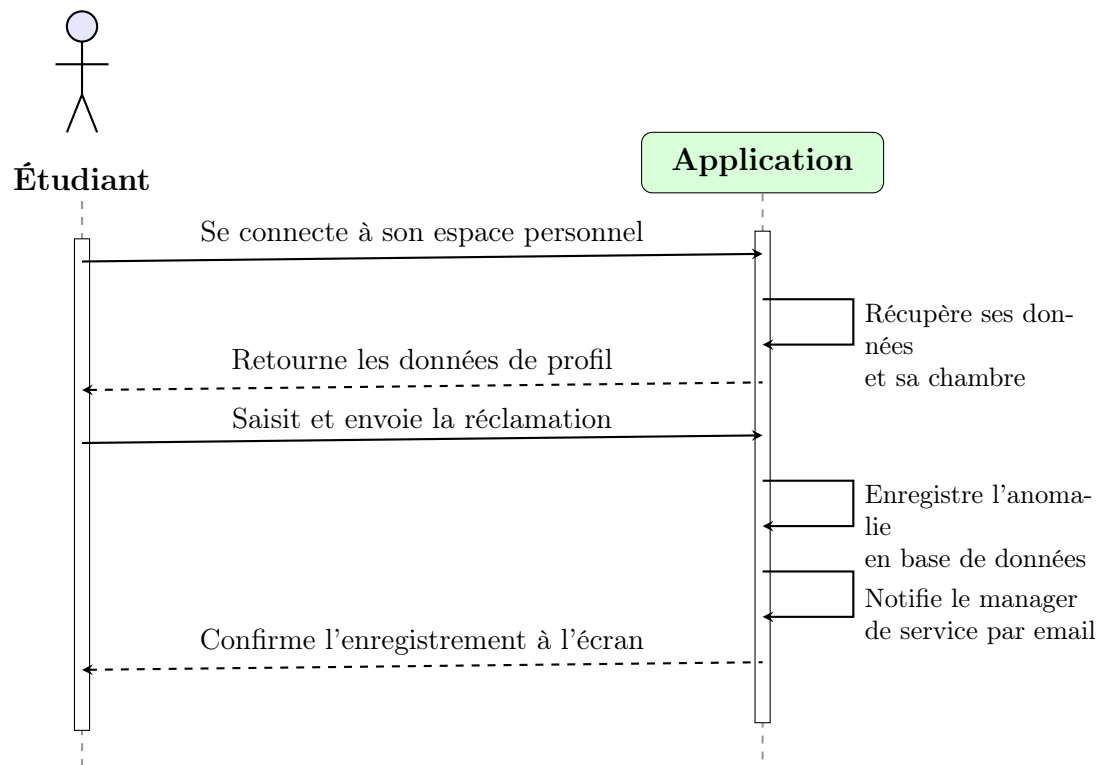
6.3 Réservation de Chambre par l'Étudiant

Ce flux permet à l'étudiant de consulter de manière transparente l'inventaire en temps réel et de réserver son affectation.



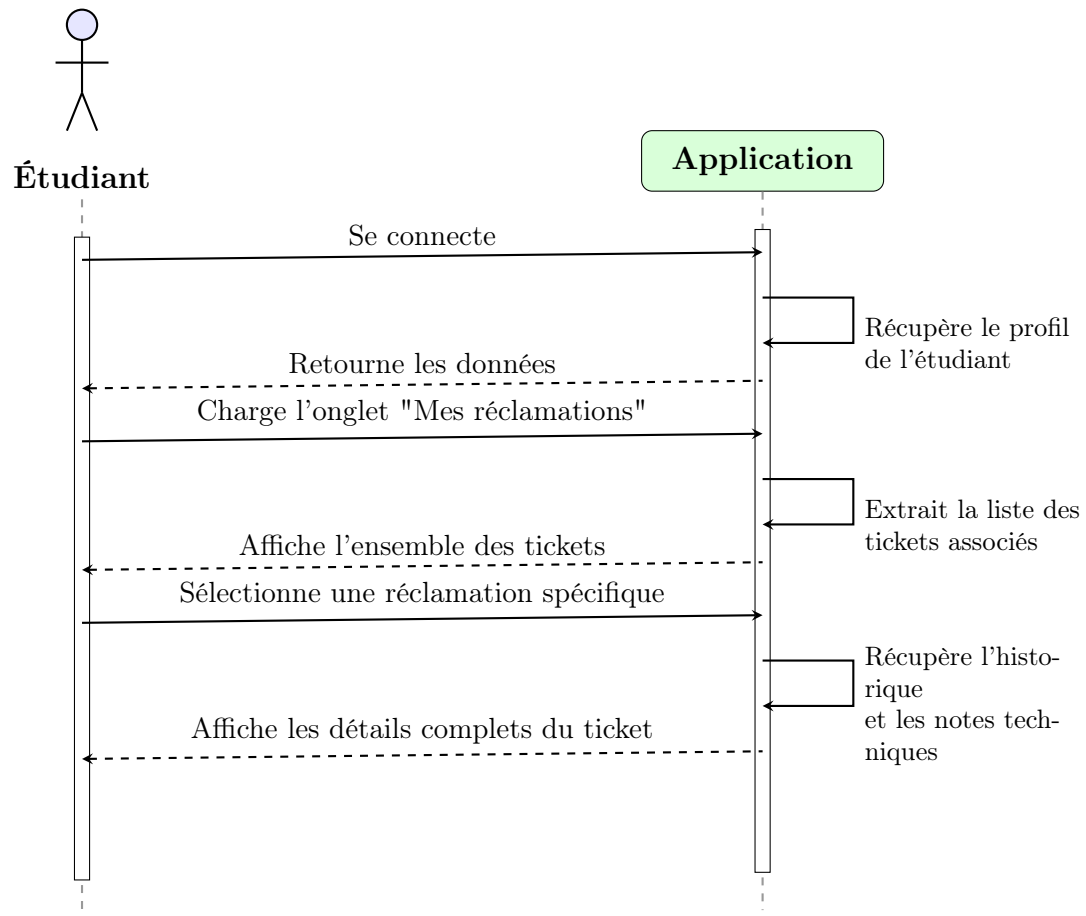
6.4 Dépôt d'une Réclamation par l'Étudiant

Ce scénario montre le flux de déclaration standardisé au sein du tableau de bord de l'application.



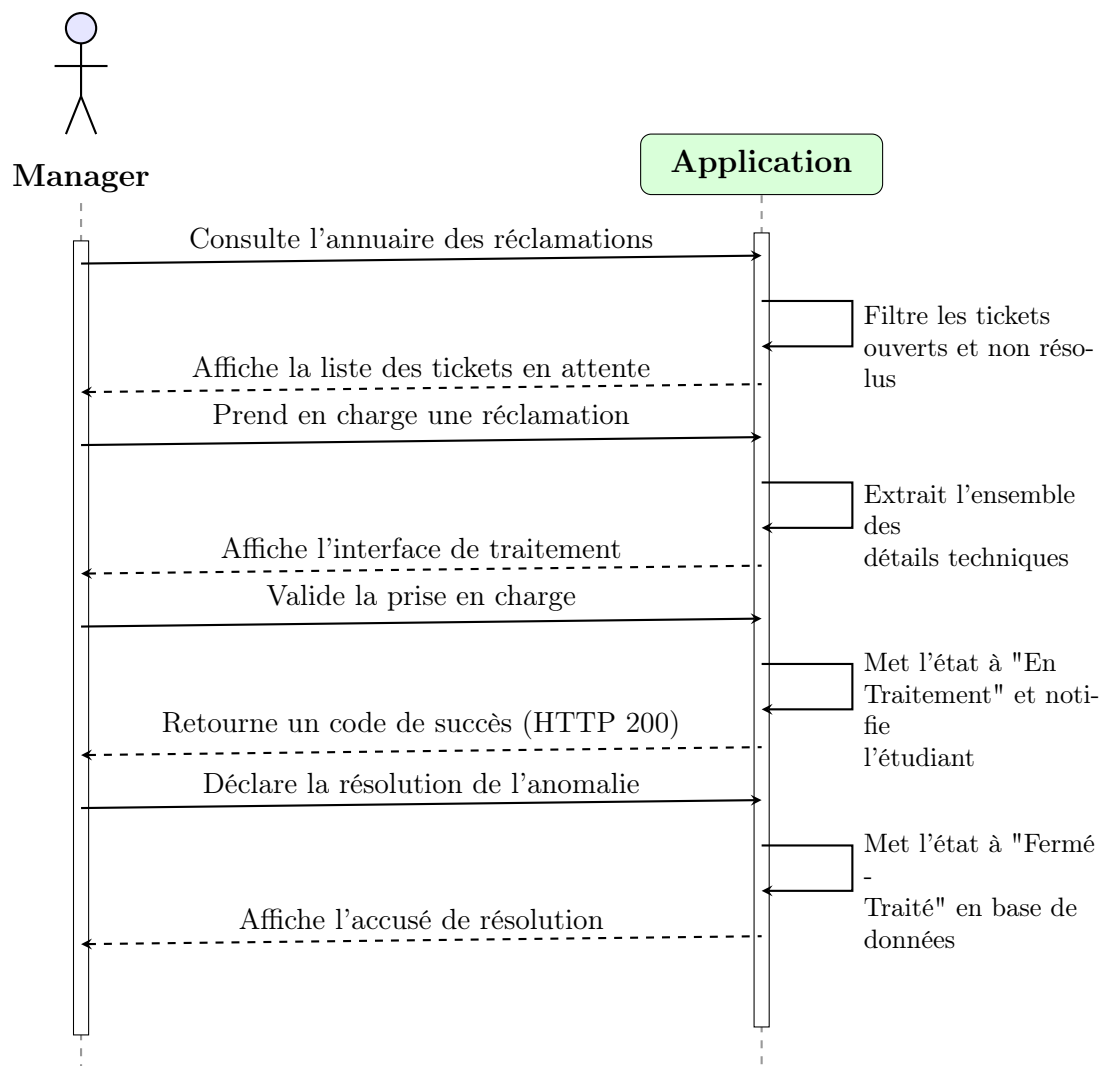
6.5 Suivi de l'Avancement par l'Étudiant

Visualisation en temps réel de l'état des tickets de maintenance ou d'assistance en cours de traitement.



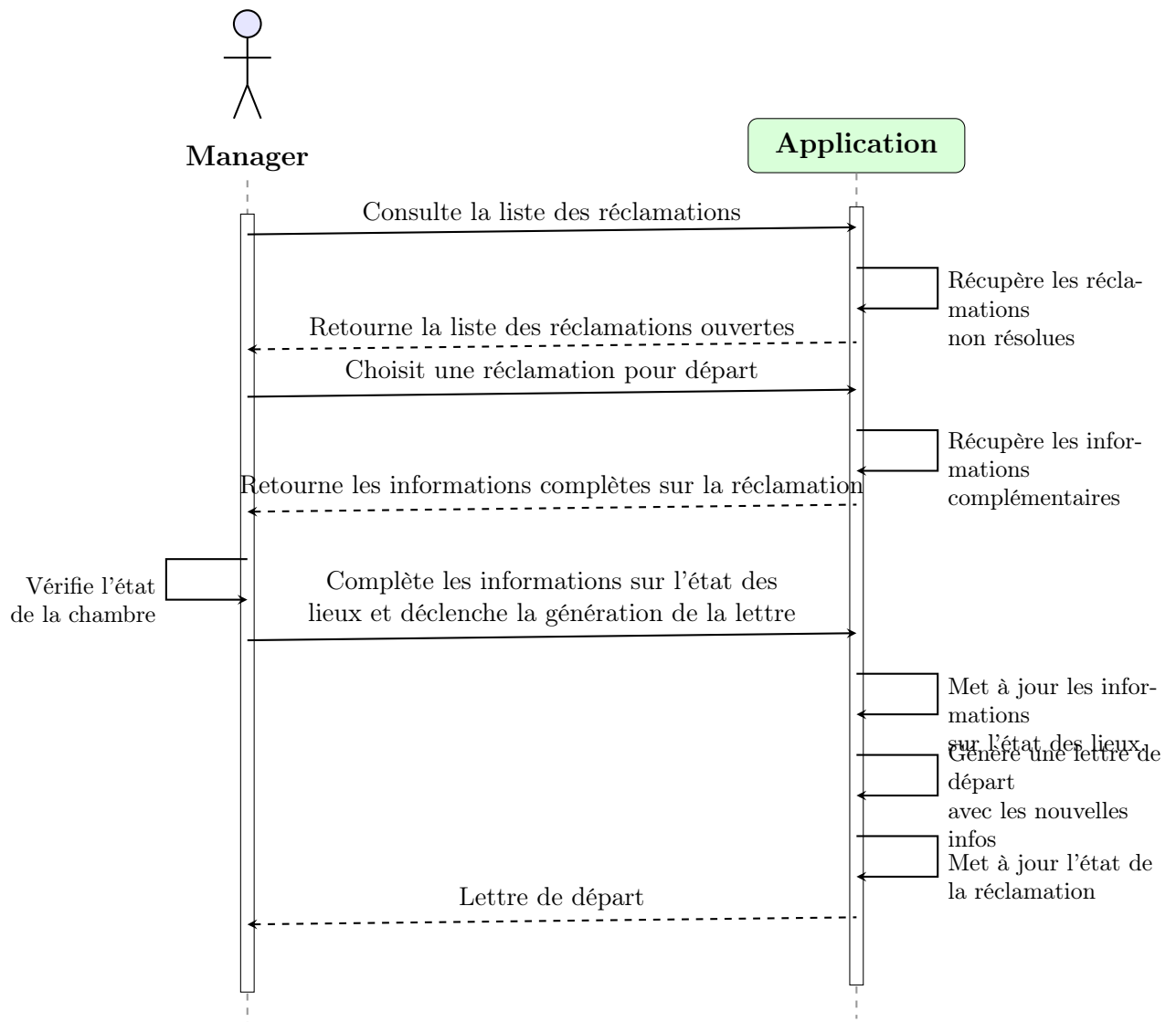
6.6 Traitement et Clôture par le Manager de Résidence

Flux opérationnel de gestion, assignation, mise à jour et résolution des réclamations par le personnel.



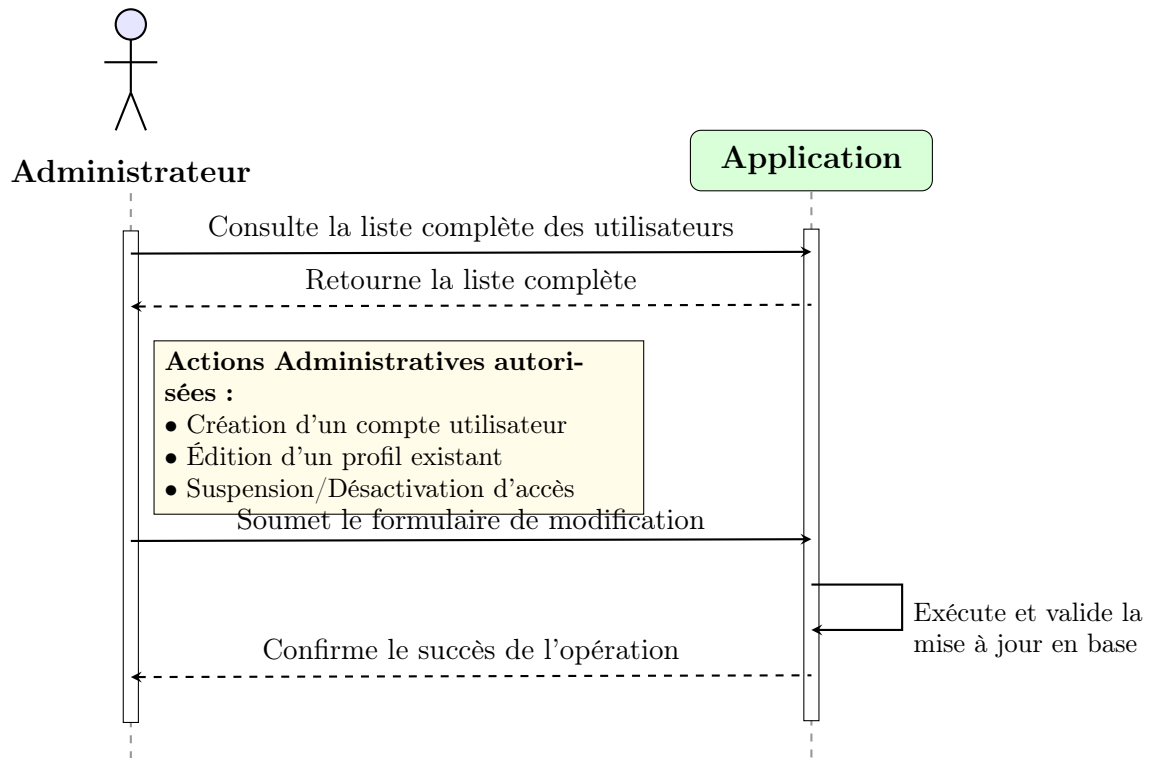
6.7 Processus Administratif de Libération de Chambre

Ce diagramme de séquence décrit la déclaration de sortie d'un résident et la validation d'état de la chambre.



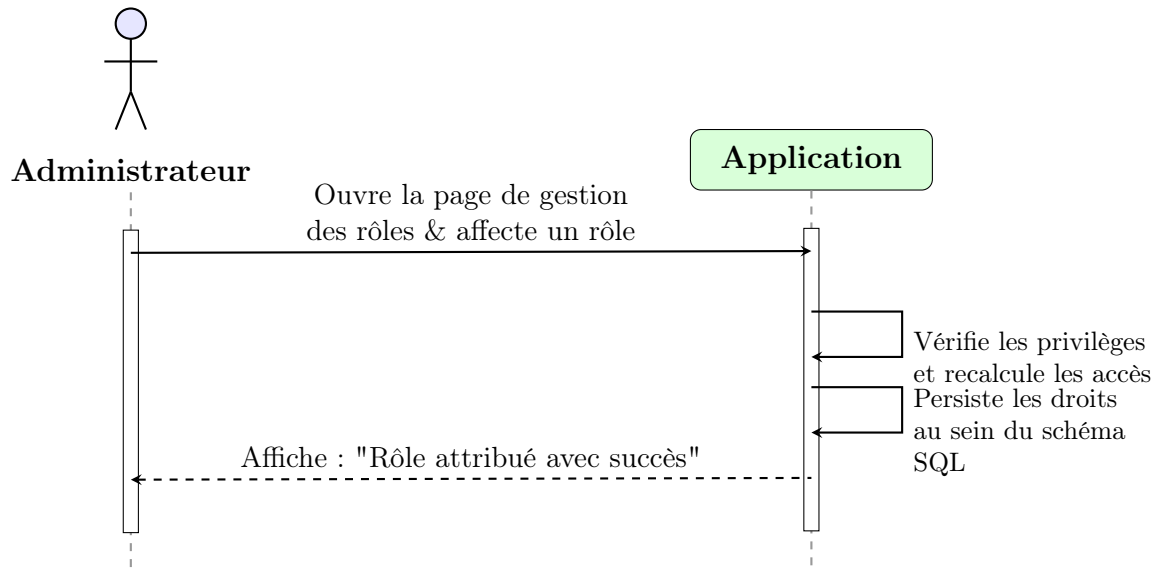
6.8 Gestion Applicative des Profils Utilisateurs

Contrôle complet des habilitations des comptes et annuaires par l'administrateur du système.



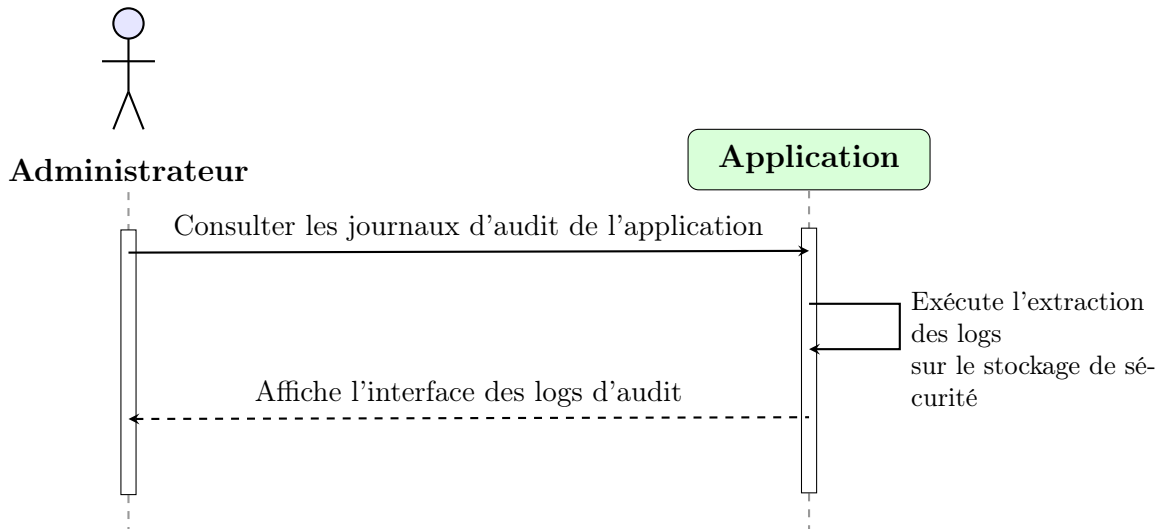
6.9 Gestion Fine des Rôles (RBAC)

Ce flux standardisé permet l'assignation et la réorganisation des rôles pour sécuriser les données et processus métiers.



6.10 Consultation des Journaux d'Audit Sécurisés

Ce diagramme modélise l'extraction sécurisée des logs techniques de traçabilité des opérations.

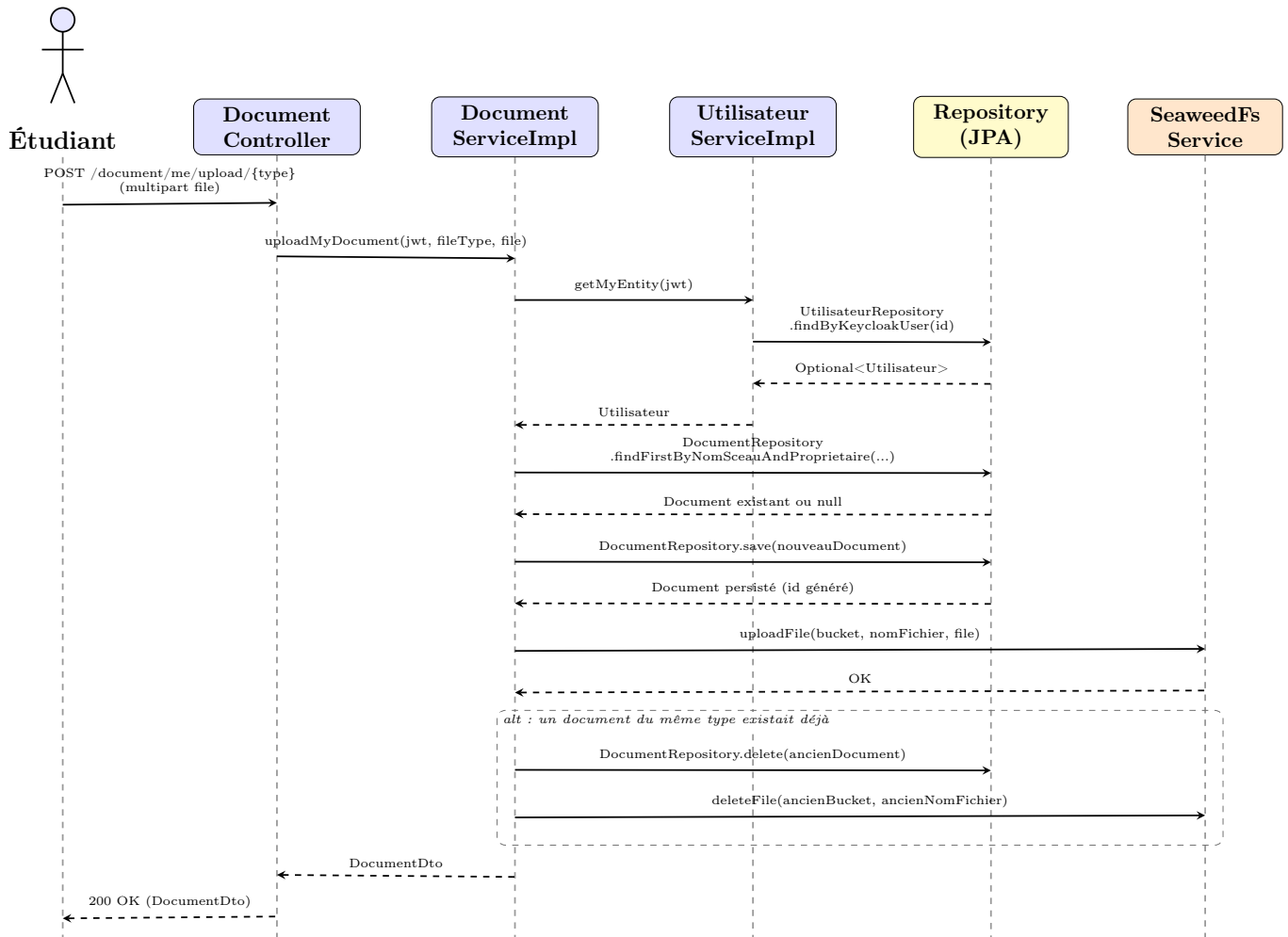


Transition vers la vue boîte ouverte. Les diagrammes boîte noire ci-dessus couvrent l'ensemble des scénarios fonctionnels prévus pour **ResiAIAC**, y compris certains scénarios non encore implémentés côté backend à ce stade (gestion des départs, suspension de compte, gestion des rôles, consultation des journaux d'audit) – ils font office de spécification cible. Alors que ces diagrammes modélisaient l'interaction externe visible de l'acteur, les diagrammes qui suivent ouvrent la boîte : chaque diagramme ci-dessous détaille, jusqu'au niveau de l'appel de méthode, le cheminement réel d'une requête à travers le Controller, la couche Service et les systèmes de persistance/stockage, tel qu'il existe effectivement dans le code source actuel. Le périmètre reprend exactement les scénarios ci-dessus qui sont implémentés côté backend : les scénarios non implémentés ci-dessus, ainsi que la distinction artificielle entre dépôt direct et dépôt depuis l'espace étudiant (fusionnée en un seul flow, les deux scénarios blackbox correspondant au même point d'entrée self-service), en sont exclus et documentés comme tels. Convention graphique : composants Spring internes en bleu, couche de

persistance en jaune, systèmes externes en orange ; les fragments alt/loop en pointillés isolent les branches conditionnelles et les boucles.

6.11 Dépôt de Documents par l'Étudiant (ouvert)

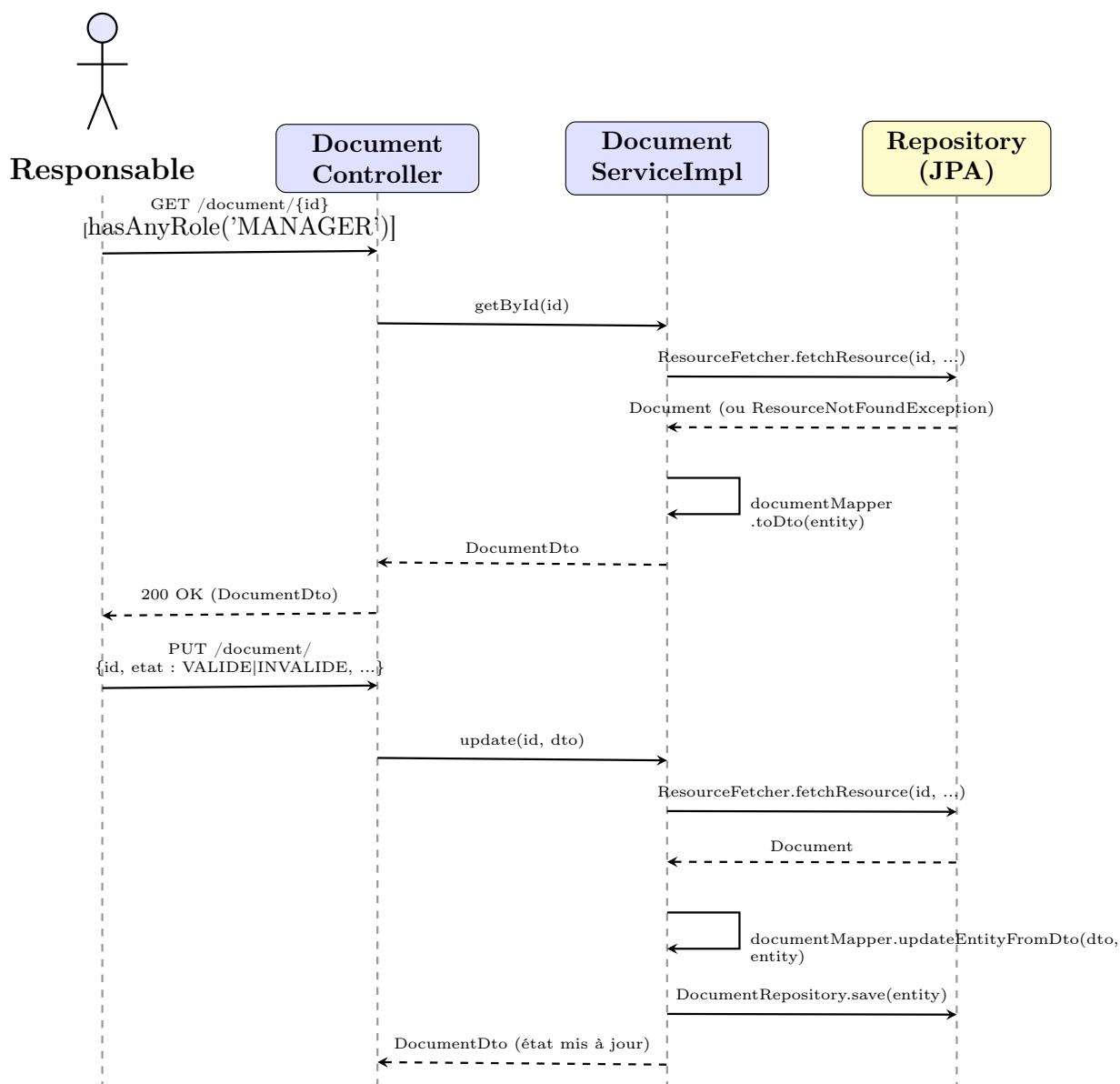
Version détaillée de ce scénario : chaque appel de méthode réel entre le contrôleur, la couche service et les systèmes de persistance/stockage est représenté, jusqu'à `DocumentServiceImpl.uploadMyDocument`.



Note : l'ordre réel du code écrit d'abord le nouveau document (persistance + upload SeaweedFS) avant de supprimer l'ancien – ce choix évite qu'un étudiant se retrouve sans document du type concerné si l'upload venait à échouer en cours de route.

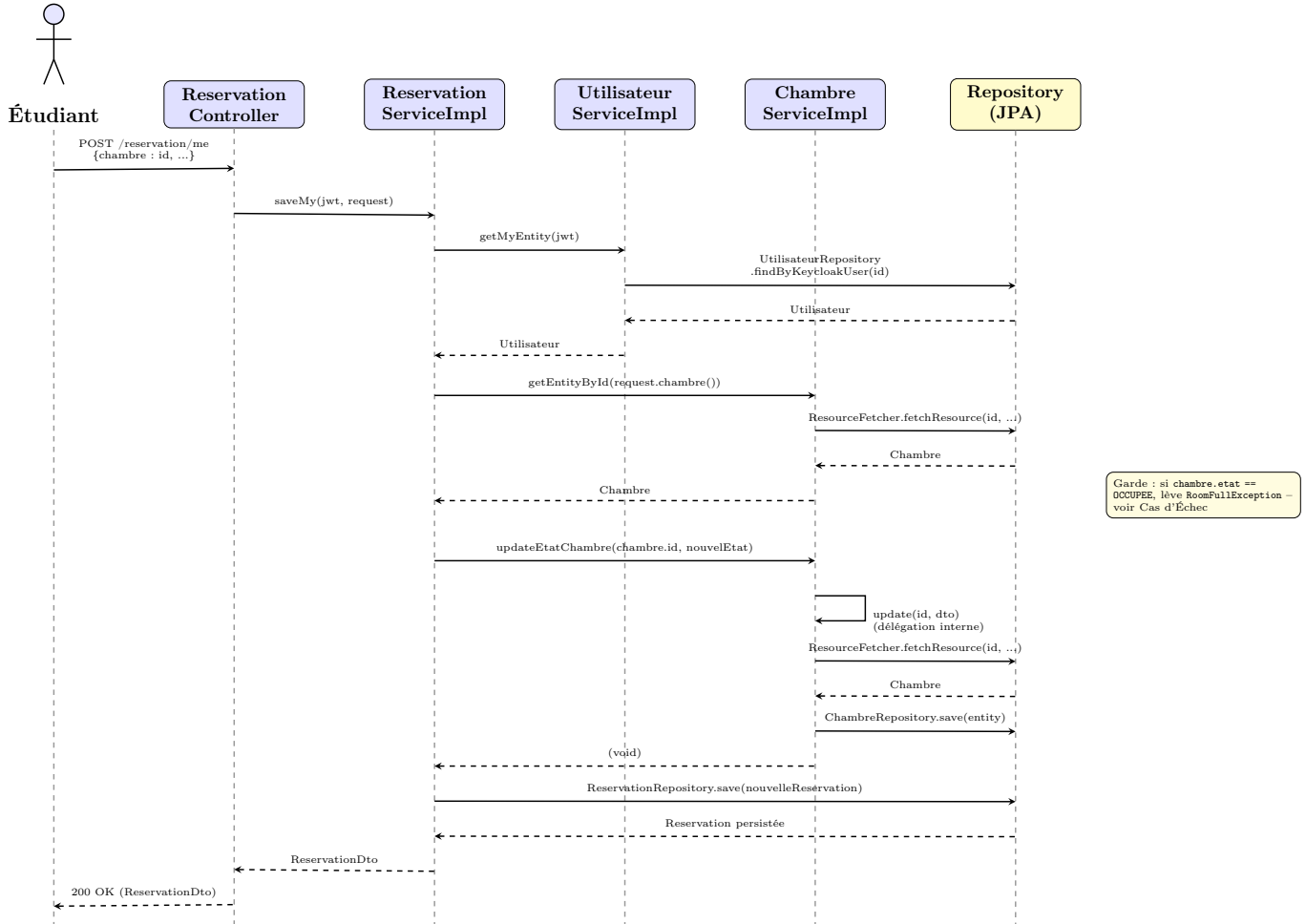
6.12 Traitement et Validation de Dossier par le Responsable (ouvert)

Version détaillée du Scénario A2. Le contrôle d'accès réel porte sur le rôle **MANAGER** (que **RESPONSABLE** hérite via la hiérarchie de rôles), et non sur un rôle **RESPONSABLE** dédié comme le nom du scénario pourrait le laisser penser.



6.13 Réserve de Chambre par l'Étudiant (ouvert)

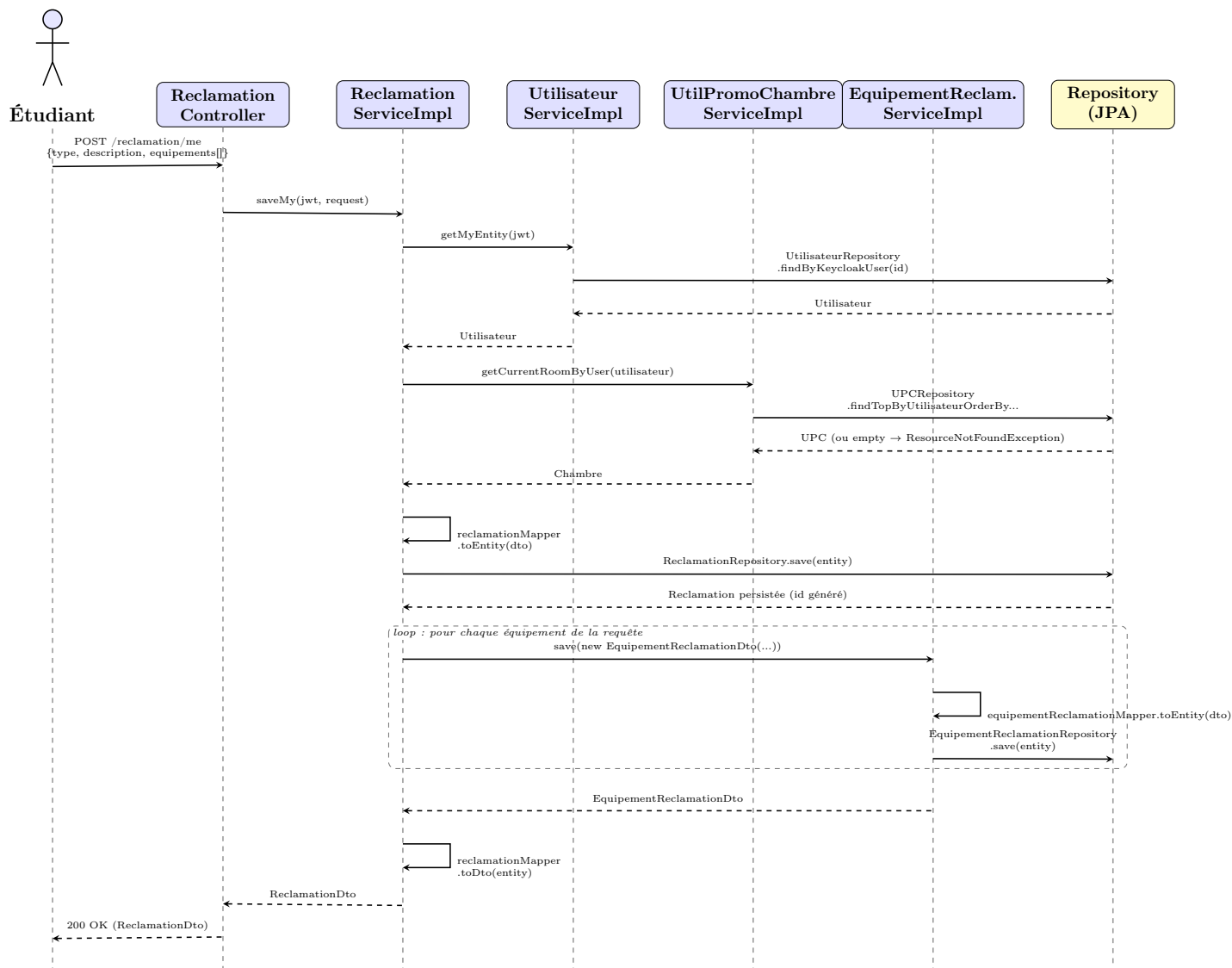
Version détaillée du Scénario B1, jusqu'à `ReservationServiceImpl.saveMy` et la délégation vers `ChambreServiceImpl` pour la transition d'état de la chambre. Le cas d'échec (chambre déjà complète) est traité séparément dans la section *Scénarios Alternatifs et Cas d'Échec*.



Note : le calcul de la nouvelle transition d'état de la chambre (LIBRE/PARTIELLEMENT_LIBRE → PARTIELLEMENT_LIBRE/OCCUPEE selon la capacité) est effectué avant même la persistance de la réservation, directement dans `saveMy`.

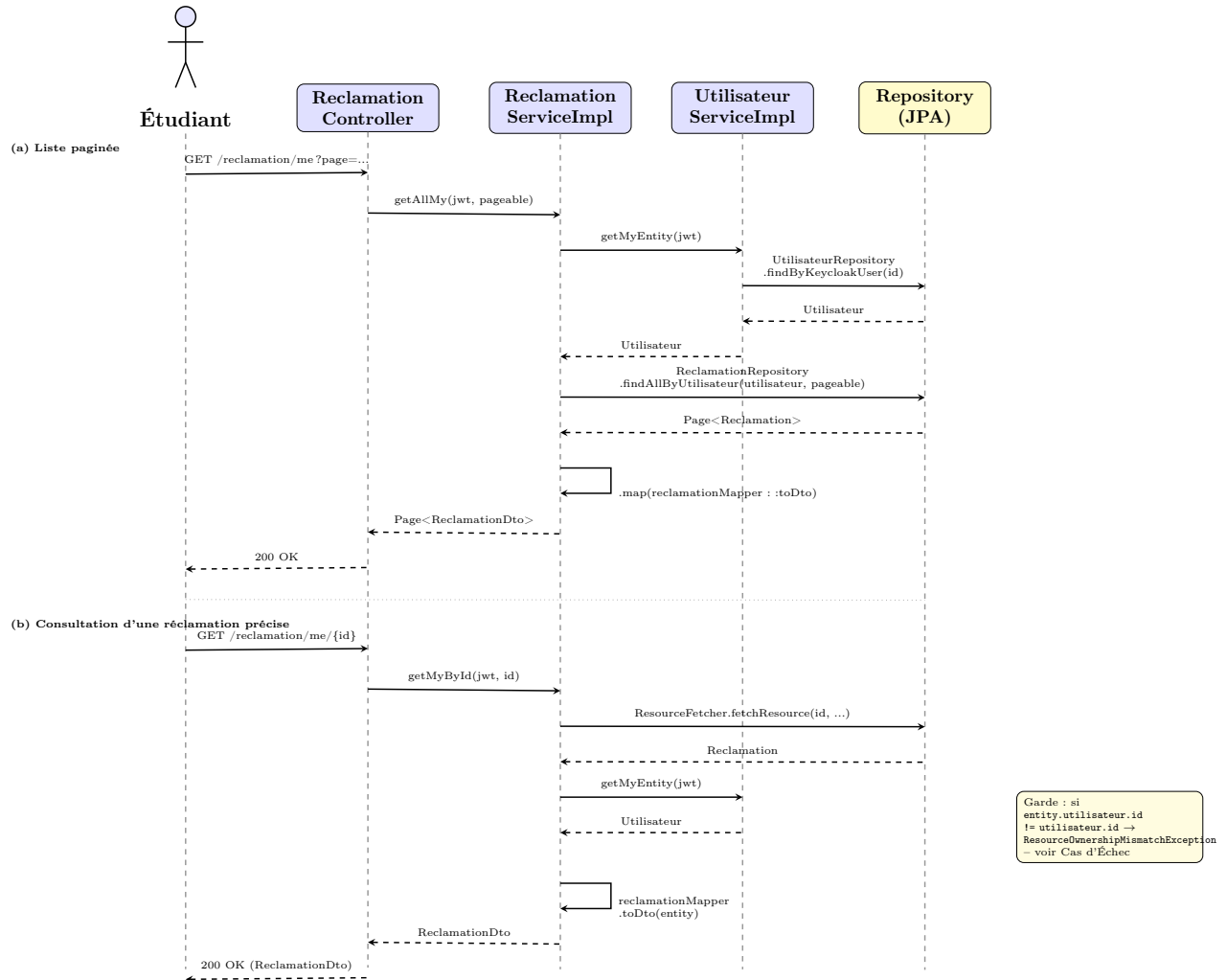
6.14 Dépôt de Réclamation par l'Étudiant (ouvert)

Les Scénarios C1 et C2 du blackbox correspondent en réalité au même unique point d'entrée self-service : `POST /reclamation/me` → `ReclamationServiceImpl.saveMy`. Ils sont donc fusionnés en un seul diagramme ouvert.



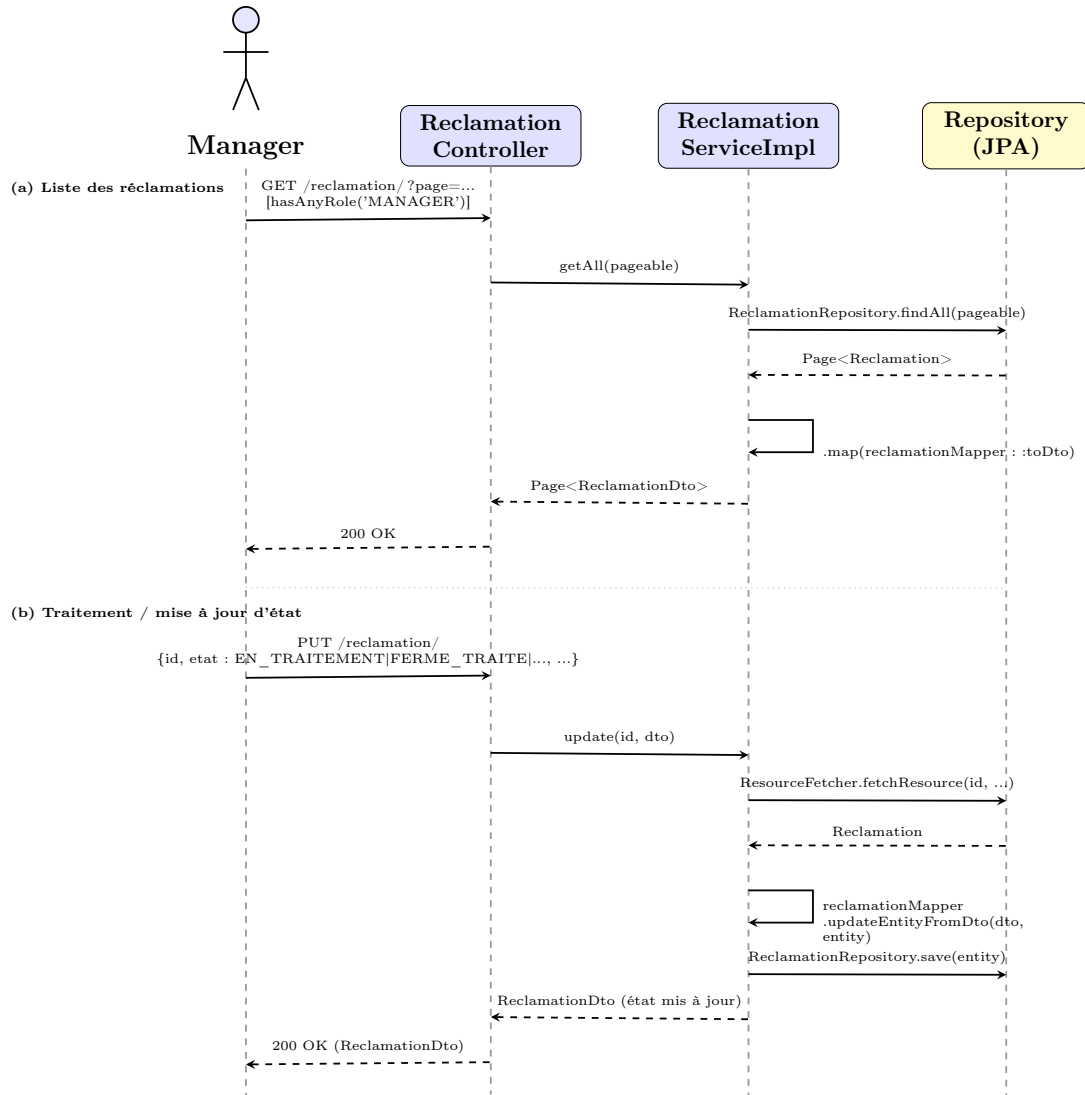
6.15 Suivi de l'Avancement par l'Étudiant (ouvert)

Version détaillée du Scénario C3, couvrant les deux routes self-service de consultation : la liste paginée (GET /reclamation/me) et la consultation d'une réclamation précise avec vérification de propriété (GET /reclamation/me/{id}).



6.16 Traitement et Clôture par le Manager de Résidence (ouvert)

Version détaillée du Scénario C4, couvrant la liste des réclamations côté gestion (GET /reclamation/) et la mise à jour d'état (PUT /reclamation/).



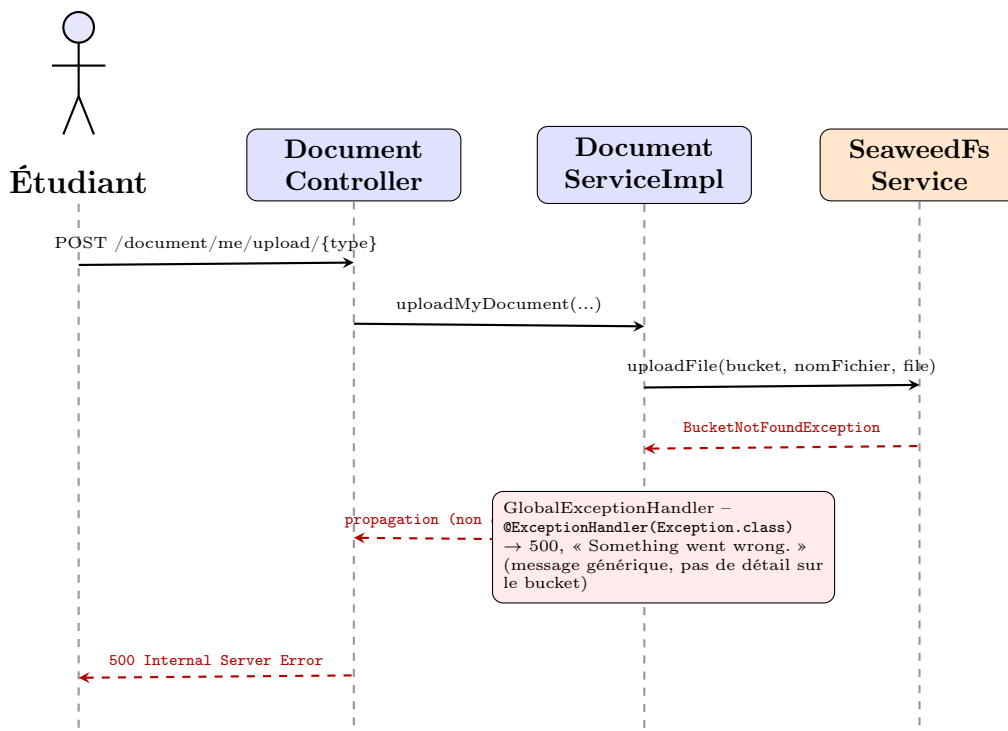
Conclusion. Ces six diagrammes couvrent l'intégralité des flux self-service et de gestion effectivement implémentés dans le backend actuel. Deux écarts avec les scénarios boîte noire d'origine ont été identifiés et corrigés au passage (fusion des deux scénarios de dépôt de réclamation en un seul, retrait du filtrage RBAC illustré pour la gestion des rôles) plutôt que reproduits tels quels. La section suivante complète cette vue par les chemins d'erreur associés à ces mêmes flux.

7 Scénarios Alternatifs et Cas d'Échec

Cette section documente les chemins d'erreur réellement gérés par le code (exceptions personnalisées interceptées par `GlobalExceptionHandler`), plutôt que des cas d'échec hypothétiques. Convention : les flèches rouges en pointillés représentent la propagation d'une exception jusqu'à sa capture.

7.1 Échec du Dépôt de Document

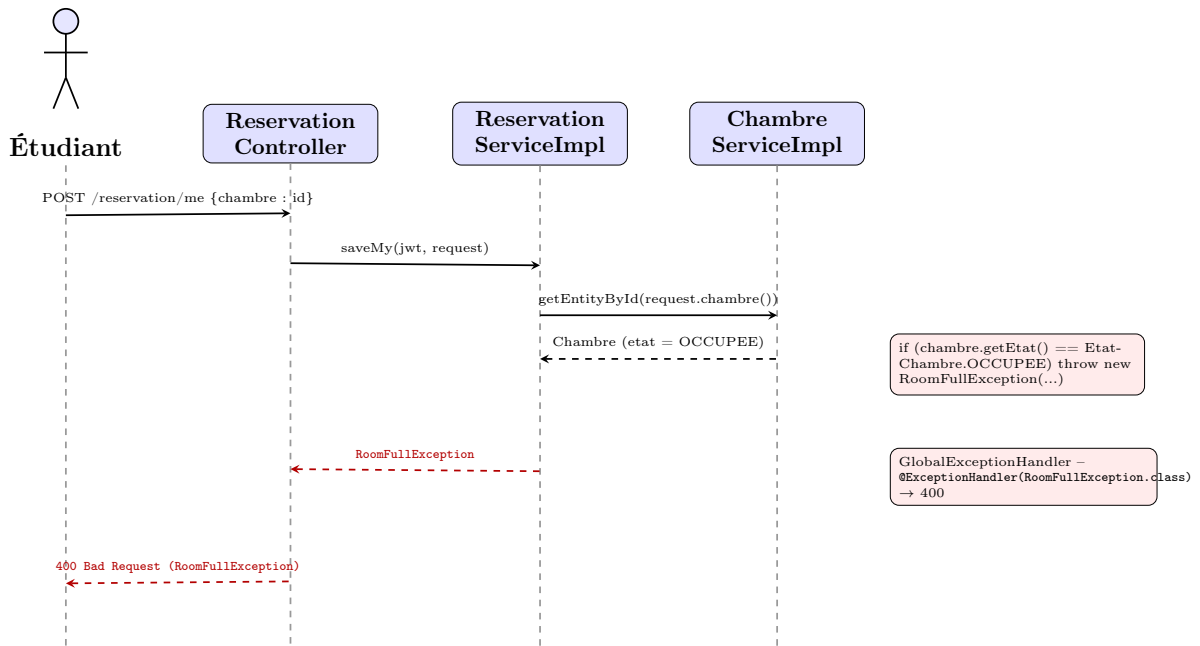
Le service `SeaweedFsService` peut lever une `BucketNotFoundException` si le bucket cible n'existe pas. Cette exception n'a **pas** de `@ExceptionHandler` dédié : elle retombe sur le gestionnaire générique `ExceptionHandler.class`, qui répond 500 avec un message générique – un gap de granularité d'erreur identifié dans le code actuel.



7.2 Rejet de Document

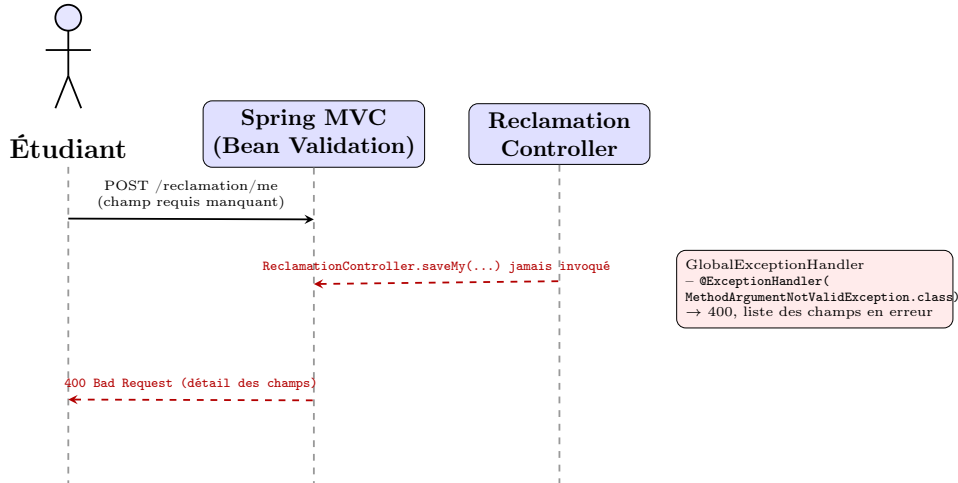
Il ne s'agit pas d'un échec technique mais d'une issue métier alternative légitime : le même appel `DocumentServiceImpl.update` est utilisé, avec `etat = INVALIDE` au lieu de `VALIDE` dans le `DocumentDto` envoyé. Aucune branche de code séparée n'existe – la validation métier (pourquoi le document est invalide) reste à la charge du responsable, hors du système.

7.3 Chambre Complète (RoomFullException)

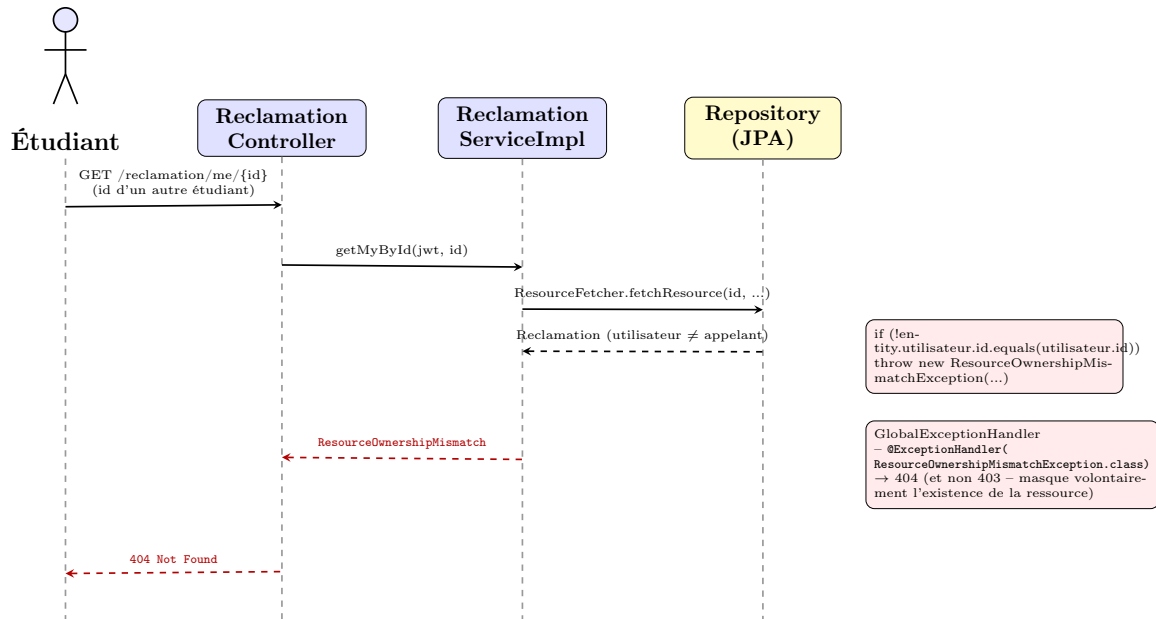


7.4 Échec de Validation d'Entrée

La validation Bean Validation (@Valid) sur ReclamationDto s'exécute au niveau du framework Spring, **avant** que le contrôleur n'invoque le service – le point d'échec n'est donc ni le contrôleur ni le service applicatif, mais l'infrastructure MVC elle-même.



7.5 Accès à la Réclamation d'un Autre Étudiant

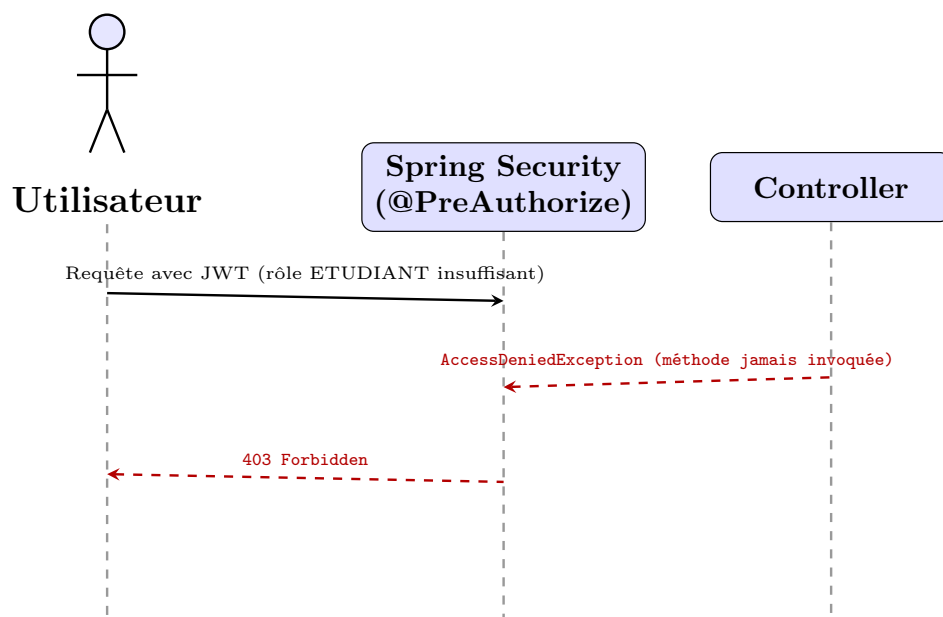


7.6 Transition d'État

Vérification faite dans le code : `ReclamationServiceImpl.update` ne comporte **aucune** garde sur la transition d'état – n'importe quelle valeur valide de l'énumération `EtatReclamation` est acceptée quel que soit l'état courant (y compris repasser une réclamation `FERME_TRAITE` à `EN_ATTENTE`). Il n'y a donc pas de cas d'échec applicatif à diagrammer ici : c'est un gap de règle métier identifié, pas une exception gérée.

7.7 Cas Transversaux : Rejet RBAC

Le contrôle `@PreAuthorize` s'exécute dans la chaîne de filtres Spring Security, **avant** d'atteindre la méthode du contrôleur – commun à tous les scénarios ci-dessus lorsque le rôle du JWT ne correspond pas.



Conclusion. L'ensemble des chemins d'erreur documentés ci-dessus correspond à des exceptions et gardes effectivement présentes dans le code : aucun cas hypothétique n'a été ajouté, et à l'inverse,

l'absence de garde constatée pour C4 a été documentée comme telle plutôt que masquée. Cette rigueur clôt la modélisation fonctionnelle du backend ; la section suivante situe ce travail dans le cycle de vie global du projet.

8 Perspectives et Périmètre du Projet

L'état actuel de **ResiAIAC** correspond à l'achèvement de sa **Phase 1 – Backend & Infrastructure** : un choix délibéré de concentrer l'effort sur la construction d'une architecture backend distribuée robuste (persistance, sécurité RBAC via Keycloak, stockage objet, gestion d'erreurs centralisée, pipeline CI/CD avec portes de qualité) avant d'étendre le périmètre applicatif. Les axes suivants sont scopés pour les phases ultérieures du cycle de vie du projet.

8.1 Frontend

Le socle backend étant désormais consolidé, le développement du frontend (SPA Angular, conformément à l'étude bibliographique) constitue la prochaine phase immédiate du projet.

8.2 Contrat d'API et Documentation

La génération dynamique d'une documentation OpenAPI/Swagger est planifiée comme amélioration future, une fois les endpoints de l'API stabilisés. Ce séquençement délibéré évite une dérive de documentation (*documentation drift*) pendant la phase actuelle de prototypage rapide, où les contrats d'API sont encore susceptibles d'évoluer.

8.3 Versionnement de Base de Données

La gestion du schéma est actuellement assurée par JPA/Hibernate (`ddl-auto: update`), un choix privilégiant la vitesse de développement durant cette phase. L'intégration d'un outil de migration versionnée tel que Flyway ou Liquibase est prévue pour la phase de mise en production, afin de garantir un versionnement immuable et traçable du schéma de base de données.

8.4 Stratégie de Mise en Cache (Redis)

Redis, déjà retenu lors de l'étude bibliographique et présent dans l'architecture de déploiement cible, n'est pas encore exploité applicativement (aucune annotation `@Cacheable` n'est présente dans le code actuel). La stratégie de mise en cache ciblera, dans une phase ultérieure, les données à faible volatilité (services, équipements) ainsi que les requêtes de lecture à haute fréquence (résolution utilisateur → Keycloak, promotion/chambre courante d'un étudiant). La séparation DTO/Entité et la centralisation des accès via la couche service ont été conçues pour que l'ajout d'un cache sur un endpoint futur reste une modification localisée, à faible friction.